

王道考研——数据结构

WWW.CSKAOYAN.COM

第三章 栈和队列

1

本节内容

栈(Stack)

基本概念

王道考研/CSKAOYAN.COM

2

知识总览

栈(Stack)

定义

基本操作

懒懒 der ...



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

王道考研/CSKAOYAN.COM

3

穿越：线性表的课件

线性表

定义

“逻辑结构”

基本操作

“运算”

懒懒 der ...



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

王道考研/CSKAOYAN.COM

4

栈的定义

线性表是具有相同数据类型的 n ($n \geq 0$) 个数据元素的有限序列，其中 n 为表长，当 $n = 0$ 时线性表是一个空表。若用 L 命名线性表，则其一般表示为

$$L = (a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表



stack

英 [stæk] 美 [stæk]

n. (整齐的) 一堆; (尤指工厂的) 大烟囱; 堆栈; (干草或谷物的) 堆; 竖着置放的高保真音响设备; 定高分层盘旋 (等待降落) 的机群; (数支步枪支起的锥形) 枪架; (车辆的) 排气管; (浪蚀) 岩柱; 垛, 堆 (木材计量单位)



王道考研/CSKAOYAN.COM

5

栈的定义

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表



王道考研/CSKAOYAN.COM

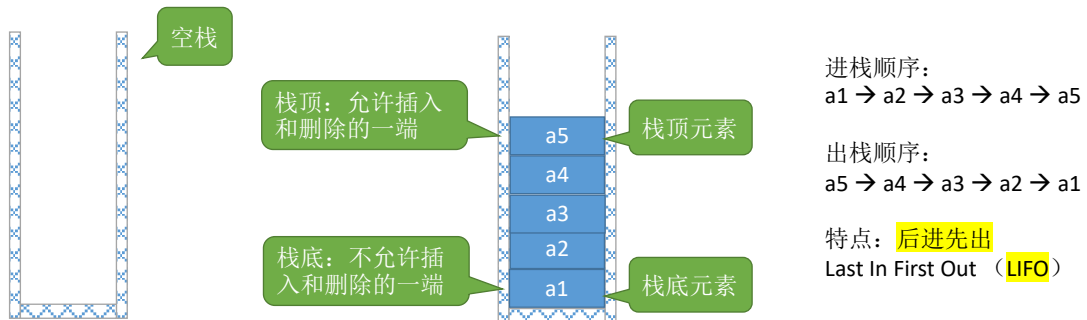
6

栈的定义

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表

逻辑结构：与普通线性表相同
数据的运算：插入、删除操作有区别

重要术语：栈顶、栈底、空栈



王道考研/CSKAOYAN.COM

7

穿越：线性表的基本操作

InitList(&L): 初始化表。构造一个空的线性表L，分配内存空间。

DestroyList(&L): 销毁操作。销毁线性表，并释放线性表L所占用的内存空间。

创、销

ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素，并用e返回删除元素的值。

增、删

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

改、查（“改”之前也要“查”）

其他常用操作：

Length(L): 求表长。返回线性表L的长度，即L中数据元素的个数。

PrintList(L): 输出操作。按前后顺序输出线性表L的所有元素值。

Empty(L): 判空操作。若L为空表，则返回true，否则返回false。



王道考研/CSKAOYAN.COM

8

栈的基本操作

InitStack(&S): 初始化栈。构造一个空栈 S, 分配内存空间。
 DestroyStack(&S): 销毁栈。销毁并释放栈 S 所占用的内存空间。

创、销

删除栈顶元素

Push(&S, x): 进栈, 若栈 S 未满, 则将 x 加入使之成为新栈顶。
 Pop(&S, &x): 出栈, 若栈 S 非空, 则弹出栈顶元素, 并用 x 返回。

增、删

不删除栈顶元素

GetTop(S, &x): 读栈顶元素。若栈 S 非空, 则用 x 返回栈顶元素

查: 栈的使用场景中大多只访问栈顶元素

其他常用操作:

StackEmpty(S): 判断一个栈 S 是否为空。若 S 为空, 则返回 true, 否则返回 false。



王道考研/CSKAOYAN.COM

9

栈的常考题型

进栈顺序:

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

有哪些合法的出栈顺序?

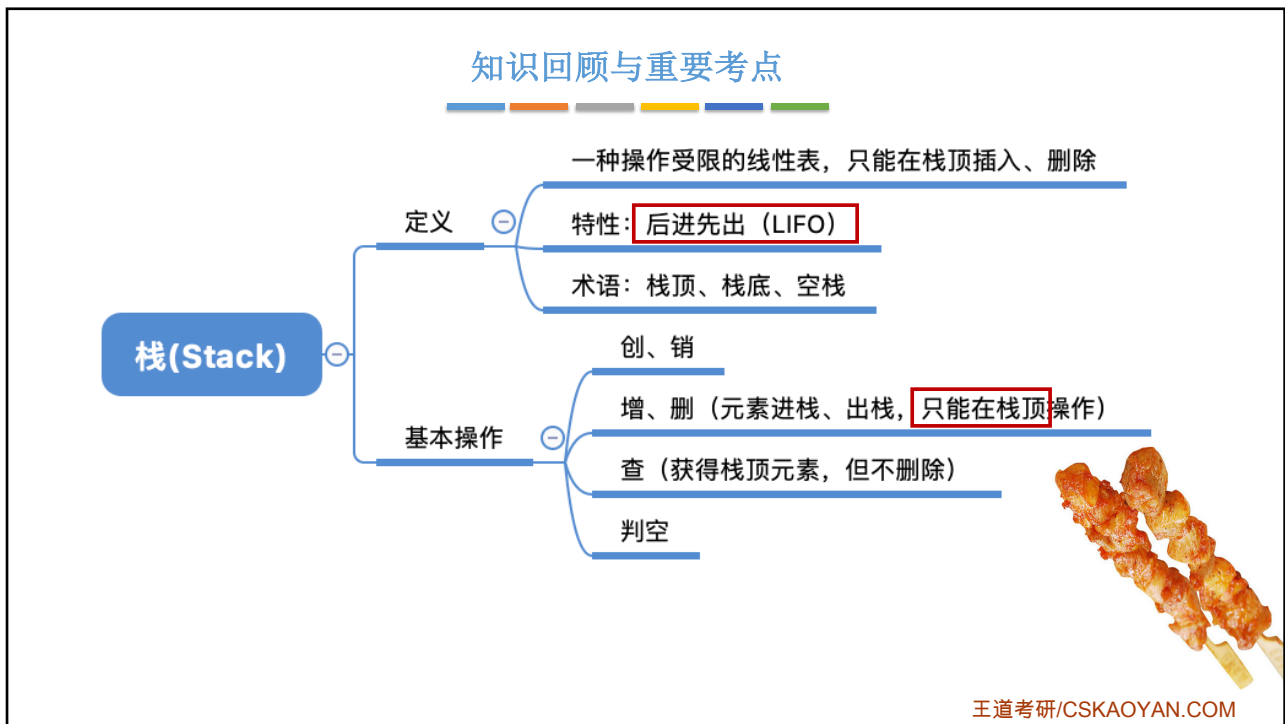


n 个不同元素进栈, 出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^n$ 。
 上述公式称为卡特兰 (Catalan) 数, 可采用数学归纳法证明 (不要求掌握)。

$$\frac{1}{5+1}C_{10}^5 = \frac{10 \times 9 \times 8 \times 7 \times 6}{6 \times 5 \times 4 \times 3 \times 2 \times 1} = 42$$

王道考研/CSKAOYAN.COM

10

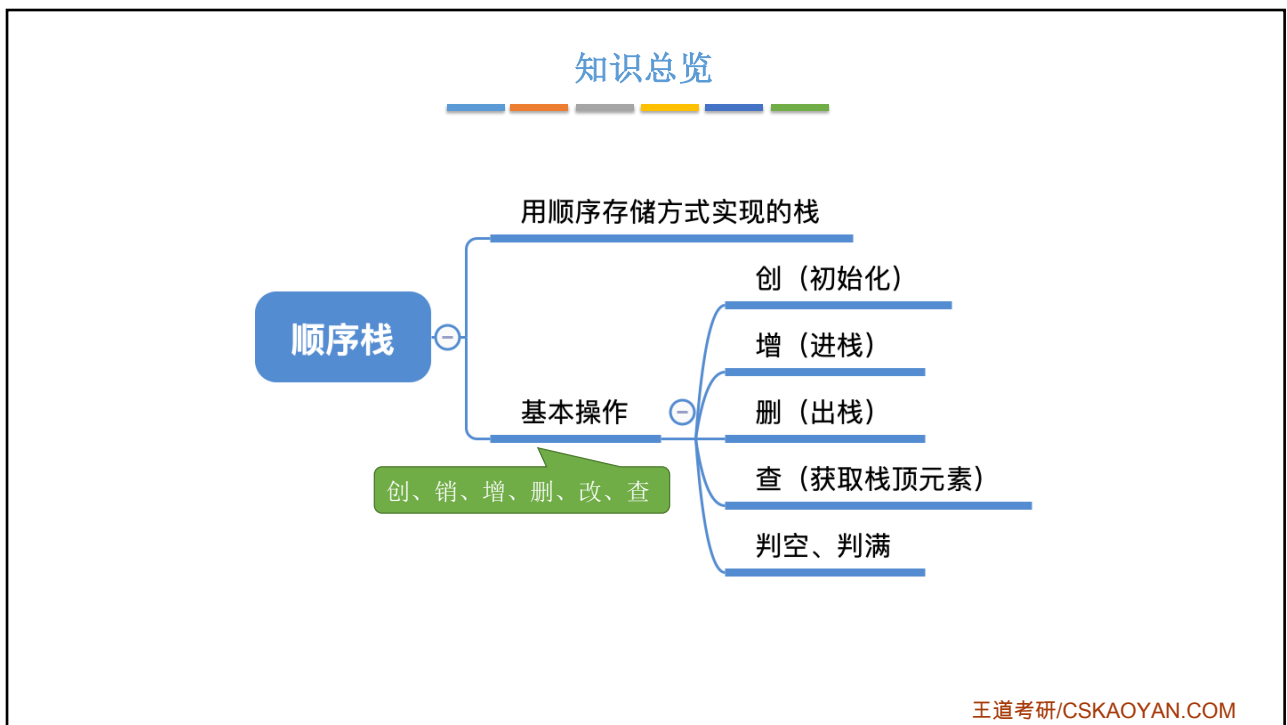


本节内容

顺序栈的实现

王道考研/CSKAOYAN.COM

1



2

顺序栈的定义

```
#define MaxSize 10
```

```
//定义栈中元素的最大个数
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
//静态数组存放栈中元素
```

```
    int top;
```

```
//栈顶指针
```

```
} SqStack;
```

Sq: sequence —— 顺序

```
void testStack() {
```

```
    → SqStack S; //声明一个顺序栈(分配空间)
```

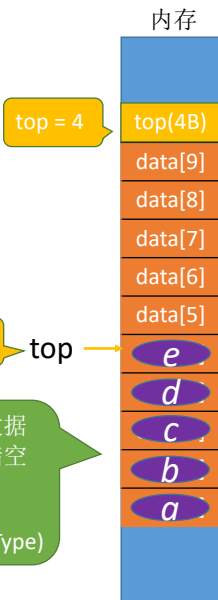
```
    //...后续操作...
```

```
}
```

top 指向栈顶元素

顺序存储: 给各个数据元素分配连续的存储空间, 大小为

MaxSize*sizeof(ElemType)



王道考研/CSKAOYAN.COM

3

初始化操作

```
#define MaxSize 10
```

```
//定义栈中元素的最大个数
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
//静态数组存放栈中元素
```

```
    int top;
```

```
//栈顶指针
```

```
} SqStack;
```

```
//初始化栈
```

```
void InitStack(SqStack &S){
```

```
    → S.top = -1;
```

```
//初始化栈顶指针
```

```
}
```

```
void testStack() {
```

```
    SqStack S; //声明一个顺序栈(分配空间)
```

```
    → InitStack(S);
```

```
    //...后续操作...
```

```
}
```

增删改查

```
//判断栈空
```

```
bool StackEmpty(SqStack S){
```

```
    if(S.top == -1)
```

```
        //栈空
```

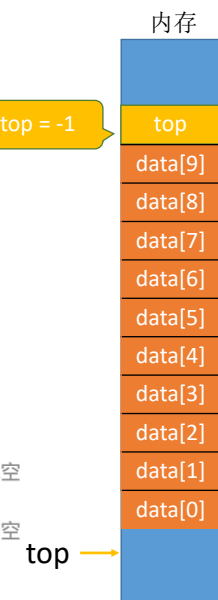
```
        return true;
```

```
    else
```

```
        //不空
```

```
        return false;
```

```
}
```



王道考研/CSKAOYAN.COM

4

进栈操作

```
#define MaxSize 10 //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize]; //静态数组存放栈中元素
    int top; //栈顶指针
} SqStack;

//新元素入栈
bool Push(SqStack &S, ElemType x){
    if(S.top==MaxSize-1) //栈满, 报错
        return false;
    S.top = S.top + 1; //指针先加1
    S.data[S.top]=x; //新元素入栈
    return true;
}
```

注意: 错误写法!

`S.data[S.top] = x;`
`S.top = S.top + 1;`

`S.data[++S.top]=x;`

真的很危险

内存

top = 1

top

data[9]

data[8]

data[7]

data[6]

data[5]

data[4]

data[3]

data[2]

b

a

top

王道考研/CSKAOYAN.COM

5

进栈操作

```
#define MaxSize 10 //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize]; //静态数组存放栈中元素
    int top; //栈顶指针
} SqStack;

//新元素入栈
bool Push(SqStack &S, ElemType x){
    if(S.top==MaxSize-1) //栈满, 报错
        return false;
    S.top = S.top + 1; //指针先加1
    S.data[S.top]=x; //新元素入栈
    return true;
}
```

注意: 错误写法!

`S.data[S.top] = x;`
`S.top = S.top + 1;`

`S.data[++S.top]=x;`

真的很危险

内存

top = 9

top

k

j

i

h

g

f

e

c

b

a

top

王道考研/CSKAOYAN.COM

6

出栈操作

```
#define MaxSize 10           //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize]; //静态数组存放栈中元素
    int top;                //栈顶指针
} SqStack;
```

```
//出栈操作
bool Pop(SqStack &S, ElemType &x){
    if(S.top == -1) //栈空, 报错
        return false;
    x = S.data[S.top]; //栈顶元素先出栈
    S.top = S.top - 1; //指针再减1
    return true;
}
```

等价

x = S.data[S.top--];

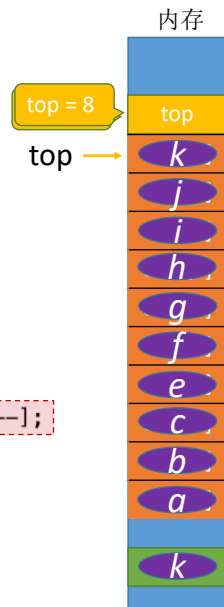
注意: 错误写法!

```
S.top = S.top - 1;
x = S.data[S.top];
```

x = S.data[--S.top];



真的很危险



数据还残留在内存中, 只是逻辑上被删除了

王道考研/CSKAOYAN.COM

7

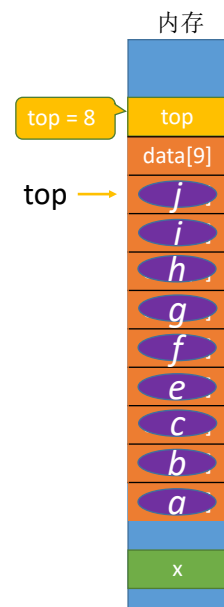
读栈顶元素操作

```
#define MaxSize 10           //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize]; //静态数组存放栈中元素
    int top;                //栈顶指针
} SqStack;
```

```
//出栈操作
bool Pop(SqStack &S, ElemType &x){
    if(S.top == -1) //栈空, 报错
        return false;
    x = S.data[S.top]; //先出栈, 指针再减1
    return true;
}
```

唯一区别

```
//读栈顶元素
bool GetTop(SqStack S, ElemType &x){
    if(S.top == -1) //栈空, 报错
        return false;
    x = S.data[S.top]; //x记录栈顶元素
    return true;
}
```



王道考研/CSKAOYAN.COM

8

另一种方式

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int top;
} SqStack;
```

//定义栈中元素的最大个数

//静态数组存放栈中元素
//栈顶指针

//初始化栈

```
void InitStack(SqStack &S){
    S.top=0;
}
```

//初始化栈顶指针

```
void testStack() {
    SqStack S; //声明一个顺序栈(分配空间)
    InitStack(S);
    //...后续操作...
}
```

//判断栈空

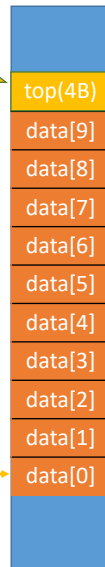
```
bool StackEmpty(SqStack S){
    if(S.top==0) //栈空
        return true;
    else //不空
        return false;
}
```

top = 0

空栈

top

内存



王道考研/CSKAOYAN.COM

9

另一种方式

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int top;
} SqStack;
```

//定义栈中元素的最大个数

//静态数组存放栈中元素
//栈顶指针

```
S.data[S.top] = x;
S.top = S.top + 1;
```

等价

```
S.data[S.top++] = x;
```

进栈

```
S.top = S.top - 1;
x = S.data[S.top];
```

```
x = S.data[--S.top];
```

出栈

注意审题 啊喂!

题目不对劲

栈满的条件: top == MaxSize

顺序栈的缺点:
栈的大小不可变

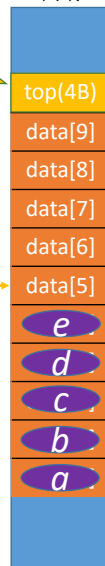
top 指向下一个
可以插入的位置

top

不会犯错



内存



王道考研/CSKAOYAN.COM

10

共享栈

两个栈共享一片空间

```

#define MaxSize 10           //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize];  //静态数组存放栈中元素
    int top0;                //0号栈栈顶指针
    int top1;                //1号栈栈顶指针
} ShStack;

//初始化栈
void InitStack(ShStack &S){
    S.top0=-1;               //初始化栈顶指针
    S.top1=MaxSize;
}

```

栈满的条件: $top0 + 1 == top1$

内存

top1 →

top0 →

王道考研/CSKAOYAN.COM

11

知识回顾与重要考点

顺序存储, 用静态数组实现, 并需要记录栈顶指针

基本操作: 创、增、删、查 — 都是 $O(1)$ 时间复杂度

两种实现:

- 初始化时 $top = -1$
 - 入栈: $S.data[++S.top] = x;$
 - 出栈: $x = S.data[S.top--];$
 - 获得栈顶元素: $x = S.data[S.top];$
 - 栈空/栈满条件是?
- 初始化时 $top = 0$
 - 入栈: $S.data[S.top++] = x;$
 - 出栈: $x = S.data[--S.top];$
 - 获得栈顶元素: $x = S.data[S.top-1];$
 - 栈空/栈满条件是?

共享栈: 两个栈共享同一内存空间, 两个栈从两边往中间增长

- 初始化: 0号栈栈顶指针初始时 $top0 = -1$; 1号栈栈顶指针初始时 $top1 = MaxSize$;
- 栈满条件: $top0 + 1 == top1$;

销? — 清空、回收

声明栈时分配内存

```

void testStack() {
    SqStack S;
    InitStack(S);
    //...后续操作...
}

```

函数运行结束后系统自动回收内存

不会犯错

王道考研/CSKAOYAN.COM

12

本节内容

链栈
的实现

王道考研/CSKAOYAN.COM

1

知识总览

链栈

用链式存储方式实现的栈

基本操作

创（初始化）
增（进栈）
删（出栈）
查（获取栈顶元素）
判空、判满

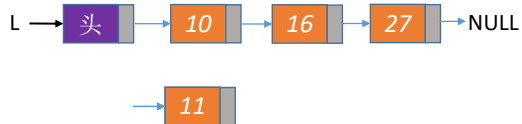
叫你偷懒

王道考研/CSKAOYAN.COM

2

穿越：头插法建立单链表

对应：进栈

对头结点的
后插操作

```

//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
  
```

叫你偷懒



头插法建立单链表：

初始化单链表

```

While 循环 {
    每次取一个数据元素 e;
    InsertNextNode (L, e);
}
  
```

王道考研/CSKAOYAN.COM

3

穿越：单链表的删除操作

对应：出栈

对头结点的
“后删”操作

又偷懒!!



王道考研/CSKAOYAN.COM

4

链栈的定义

```
typedef struct Linknode{
    ElemType data;           //数据域
    struct Linknode *next;   //指针域
} *LiStack;                 //栈类型定义
```

带头结点的初始化

S → 头 → NULL

S → 头 → d → c → b → a → NULL

不带头结点的初始化

S → NULL

S → d → c → b → a → NULL

进栈/出栈都只能在栈顶一端进行
(链头作为栈顶)

王道考研/CSKAOYAN.COM

5

知识回顾与重要考点

链栈

用链式存储方式实现的栈

两种实现方式

- 带头结点
- 不带头结点 (推荐)

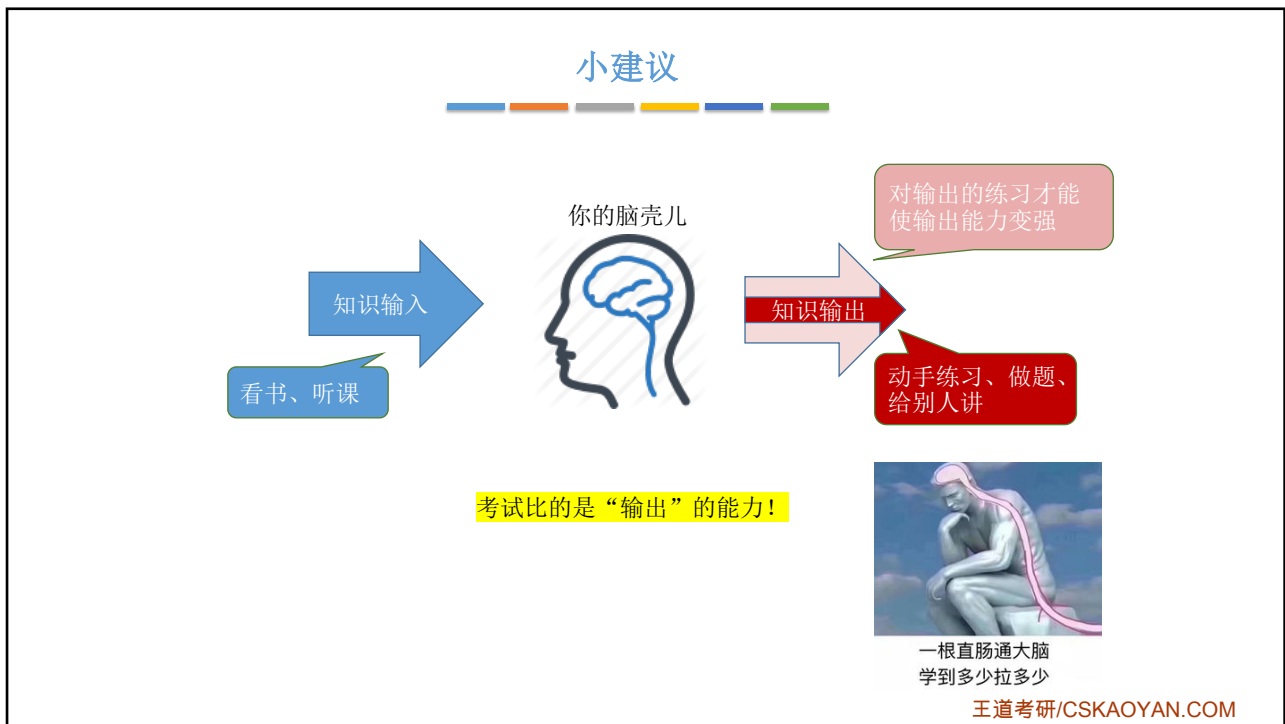
重要基本操作

- 创 (初始化)
- 增 (进栈)
- 删 (出栈)
- 查 (获取栈顶元素)
- 如何判空、判满?

动手写一遍

王道考研/CSKAOYAN.COM

6



7

本节内容

队列
基本概念

王道考研/CSKAOYAN.COM

1

知识总览

队列(Queue)

定义

基本操作

懒懒 der...

注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

王道考研/CSKAOYAN.COM

2

队列的定义

线性表是具有相同数据类型的 n ($n \geq 0$) 个数据元素的有限序列，其中 n 为表长，当 $n = 0$ 时线性表是一个空表。若用 L 命名线性表，则其一般表示为

$$L = (a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

进栈

出栈

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表

队列 (Queue) 是只允许在一端进行插入，在另一端删除的线性表

入队

出队



王道考研/CSKAOYAN.COM

3

队列的定义

队列 (Queue) 是只允许在一端进行插入，在另一端删除的线性表



特点：先进入队列的元素先出队

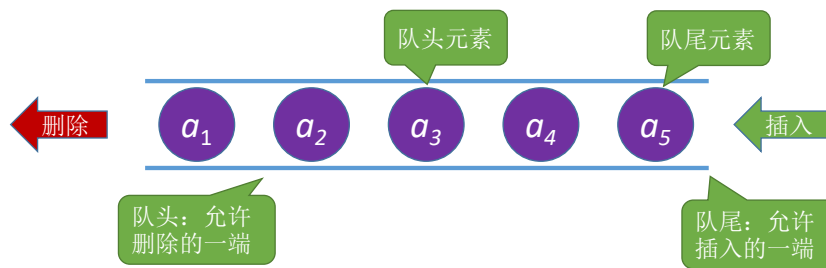
王道考研/CSKAOYAN.COM

4

队列的定义

队列 (Queue) 是只允许在一端进行插入，在另一端删除的线性表

重要术语：队头、队尾、空队列



队列的特点：先进先出
First In First Out (FIFO)

王道考研/CSKAOYAN.COM

5

队列的基本操作

InitQueue(&Q): 初始化队列，构造一个空队列Q。

DestroyQueue(&Q): 销毁队列。销毁并释放队列Q所占用的内存空间。

创、销

删除队头元素

EnQueue(&Q,x): 入队，若队列Q未满，将x加入，使之成为新的队尾。

DeQueue(&Q,&x): 出队，若队列Q非空，删除队头元素，并用x返回。

增、删

不删除队头元素

GetHead(Q,&x): 读队头元素，若队列Q非空，则将队头元素赋值给x。

查：队列的使用场景中大多只访问队头元素

其他常用操作：

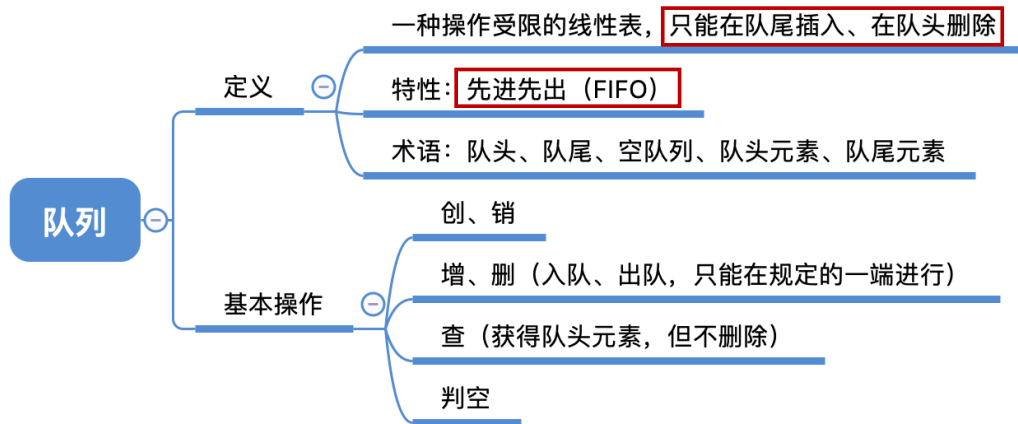
QueueEmpty(Q): 判队列空，若队列Q为空返回true，否则返回false。



王道考研/CSKAOYAN.COM

6

知识回顾与重要考点



王道考研/CSKAOYAN.COM

本节内容

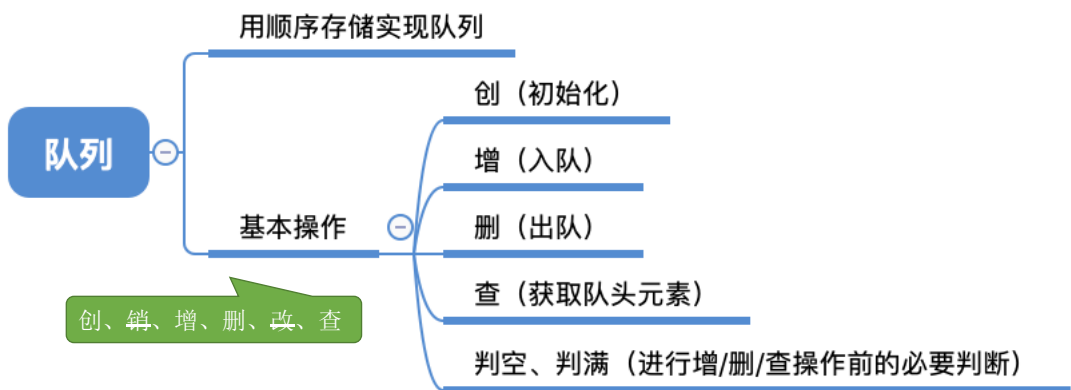
队列

顺序实现

王道考研/CSKAOYAN.COM

1

知识总览



王道考研/CSKAOYAN.COM

2

队列的顺序实现

```
#define MaxSize 10
```

```
typedef struct{
```

```
ElemType data[MaxSize];
```

```
int front, rear;
```

```
} SqQueue;
```

Sq: sequence —— 顺序

rear

英 [riə(r)] 美 [riə]

n. 后面; 后方部队; 屁股
adj. 后方的, 后面的; 背面的

```
void testQueue(){
```

```
    SqQueue Q; //声明一个队列 (顺序存储)
```

```
    //...后续操作...
```

```
}
```

```
//定义队列中元素的最大个数
```

```
//用静态数组存放队列元素
```

```
//队头指针和队尾指针
```

连续的存储空间, 大小
MaxSize*sizeof(ElemType)

front

英 [frʌnt] 美 [frʌnt]

n. 前面; 正面; 前线
vt. 面对; 朝向; 对付
vi. 朝向
adj. 前面的; 正面的

指向队尾元素的
后一个位置
(下一个应该
插入的位置)

rear

指向队
头元素

front

内存

0

front

5

rear

data[9]

data[8]

data[7]

data[6]

data[5]

e

d

c

b

a

王道考研/CSKAOYAN.COM

3

初始化操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
ElemType data[MaxSize];
```

```
int front, rear;
```

```
} SqQueue;
```

```
//初始化队列
```

```
void InitQueue(SqQueue &Q){
```

```
    //初始时 队头、队尾指针指向0
```

```
    Q.rear=Q.front=0;
```

```
}
```

```
void testQueue(){
```

```
    //声明一个队列 (顺序存储)
```

```
    SqQueue Q;
```

```
    InitQueue(Q);
```

```
    //...后续操作...
```

```
}
```

增删改查

```
//定义队列中元素的最大个数
```

```
//用静态数组存放队列元素
```

```
//队头指针和队尾指针
```

```
//判断队列是否为空
```

```
bool QueueEmpty(SqQueue Q){
```

```
    if(Q.rear==Q.front) //队空条件
```

```
        return true;
```

```
    else
```

```
        return false;
```

```
}
```

内存

0

front

1

rear

data[9]

data[8]

data[7]

data[6]

data[5]

data[4]

data[3]

data[2]

data[1]

a

rear

front

王道考研/CSKAOYAN.COM

4

入队操作 只能从队尾入队（插入）

```

#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;

//入队
bool EnQueue(SqQueue &Q, ElemType x){
    if(队列已满)
        return false; //队满则报错
    Q.data[Q.rear]=x; //将x插入队尾
    Q.rear=Q.rear+1; //队尾指针后移
    return true;
}

```

//定义队列中元素的最大个数
//用静态数组存放队列元素
//队头指针和队尾指针

队列已满的条件：
rear==MaxSize ???

内存

王道考研/CSKAOYAN.COM

5

入队操作 只能从队尾入队（插入）

```

#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;

//入队
bool EnQueue(SqQueue &Q, ElemType x){
    if(队列已满)
        return false; //队满则报错
    Q.data[Q.rear]=x; //新元素插入队尾
    Q.rear=(Q.rear+1)%MaxSize; //队尾指针加1取模
    return true;
}

```

//定义队列中元素的最大个数
//用静态数组存放队列元素
//队头指针和队尾指针

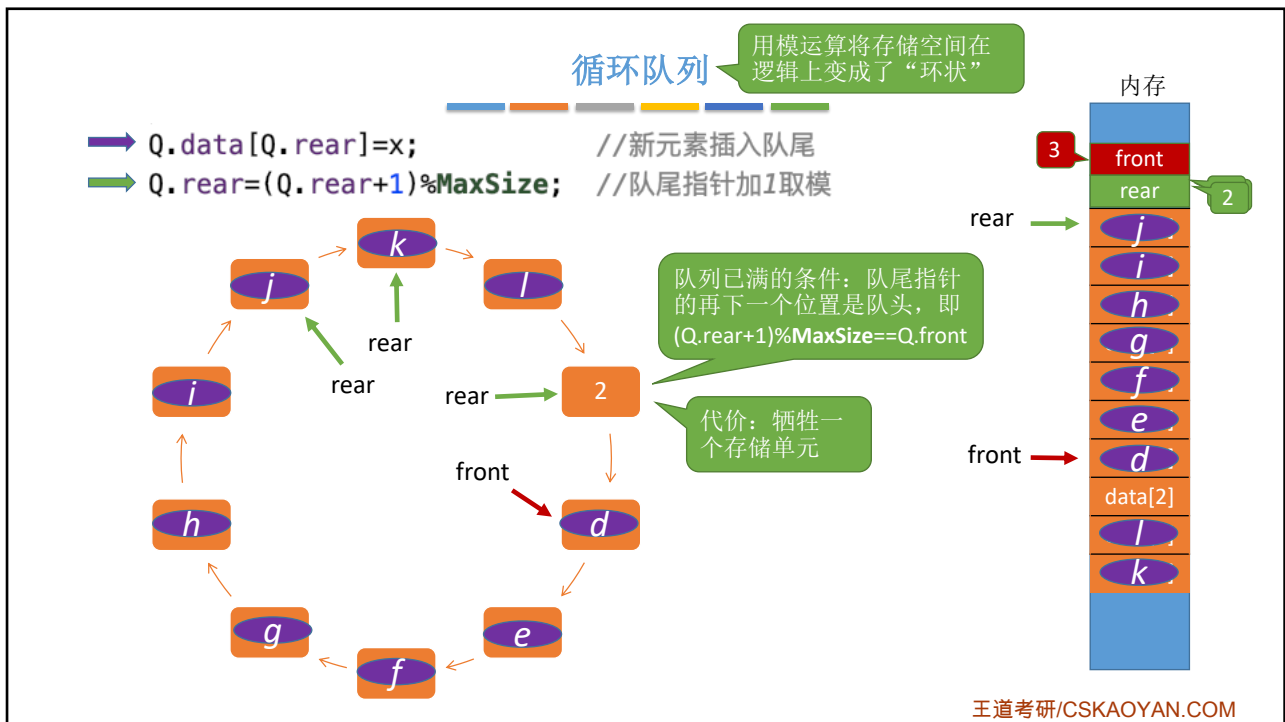
模运算将无限的整数域映射到有限的整数集合 $\{0, 1, 2, \dots, b-1\}$ 上

跨考Tips: 取模运算, 即取余运算。两个整数 a, b , $a \% b == a$ 除以 b 的余数
在《数论》中, 通常表示为 $a \text{ MOD } b$

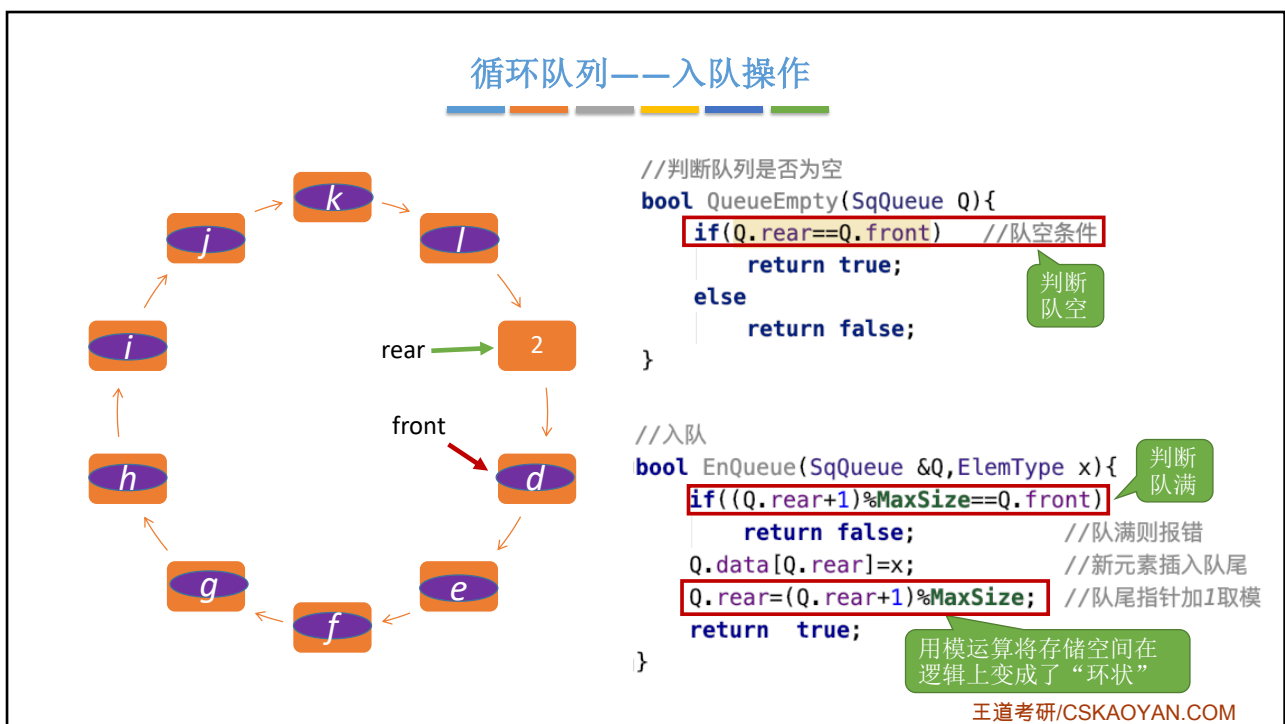
内存

王道考研/CSKAOYAN.COM

6



7



8

循环队列——出队操作

只能让队头元素出队

```
//出队（删除一个队头元素，并用x返回）
bool DeQueue(SqQueue &Q,ElemType &x){
    if(Q.rear==Q.front) return false; //队空则报错
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;
    return true;
}

//获得队头元素的值，用x返回
bool GetHead(SqQueue Q,ElemType &x){
    if(Q.rear==Q.front) return false; //队空则报错
    x=Q.data[Q.front];
    return true;
}
```

王道考研/CSKAOYAN.COM

9

队列元素个数：
 $(\text{rear} + \text{MaxSize} - \text{front}) \% \text{MaxSize}$

方案一：判断队列已满/已空

出题老师：不准浪费这可怜孤独的存储空间!!!

初始化时 rear=front=0

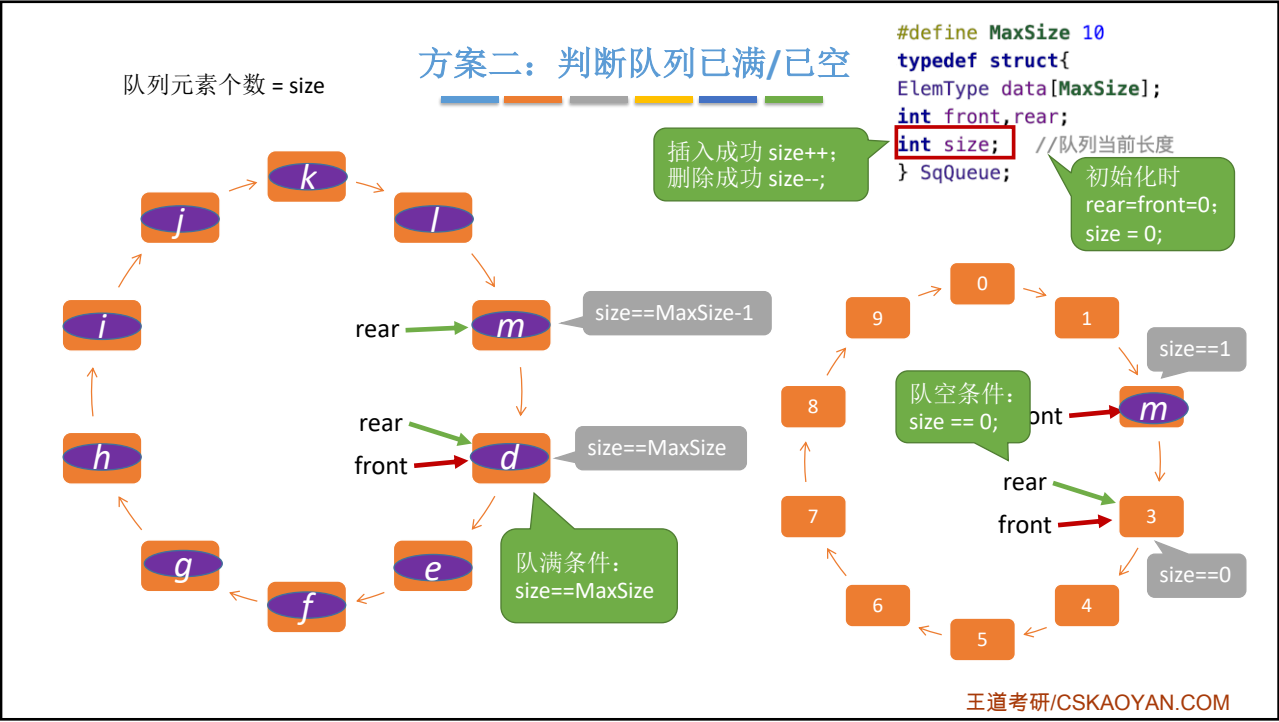
队空条件：
 $\text{Q.rear} == \text{Q.front}$

队列已满的条件：队尾指针的再下一个位置是队头，即
 $(\text{Q.rear} + 1) \% \text{MaxSize} == \text{Q.front}$

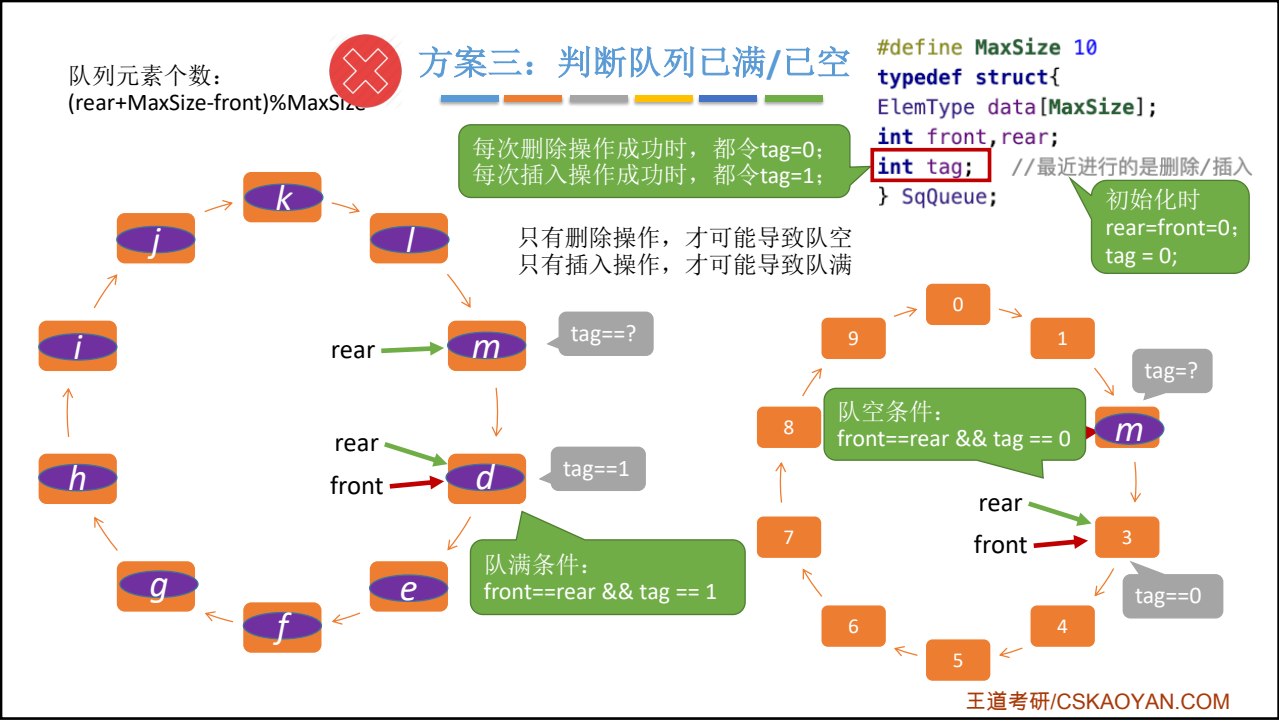
```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```

王道考研/CSKAOYAN.COM

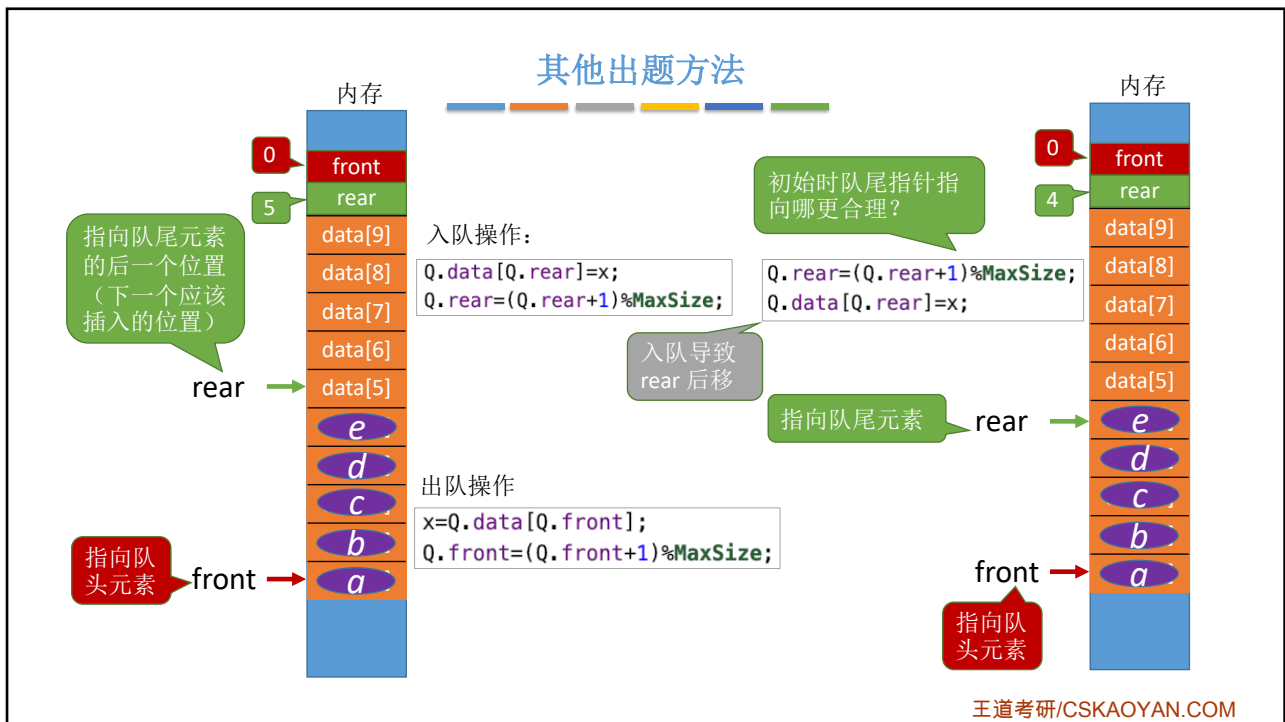
10



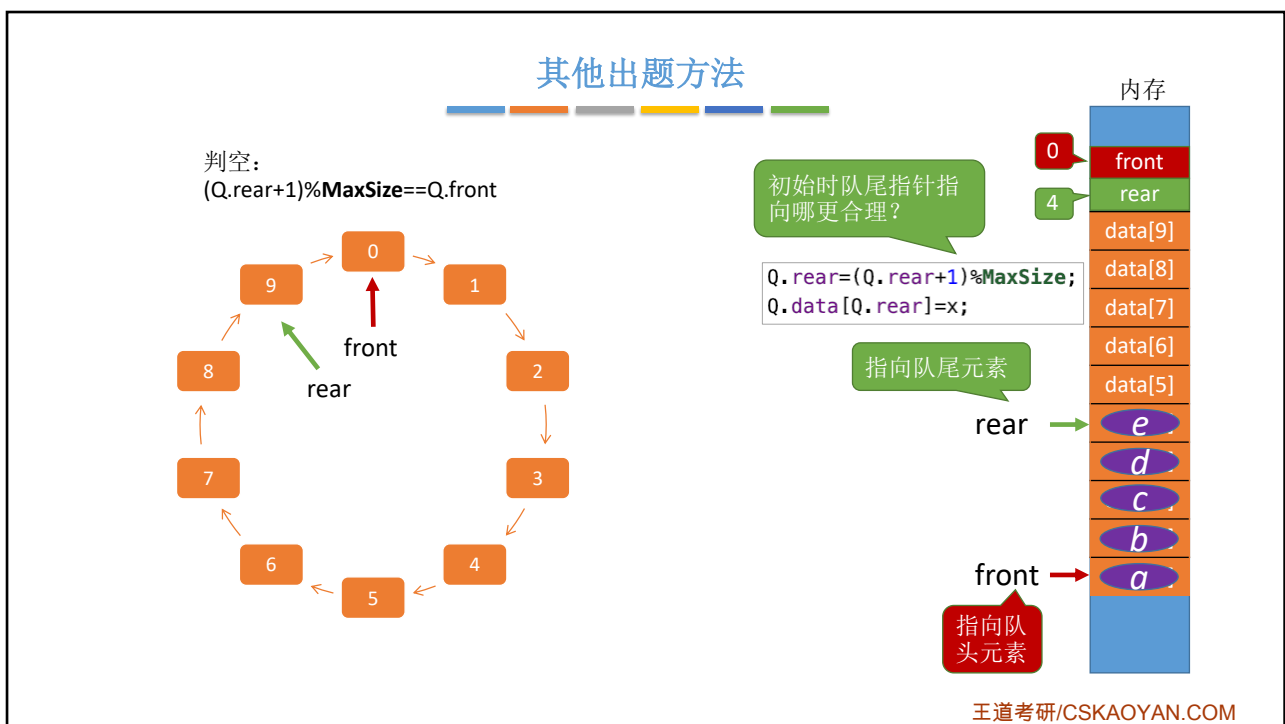
11



12



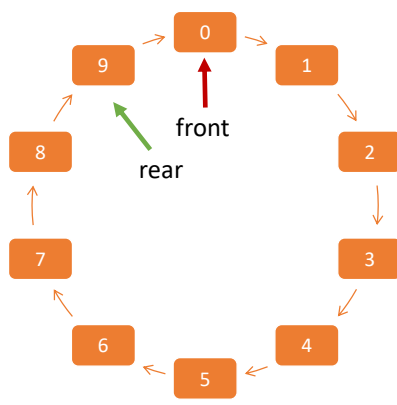
13



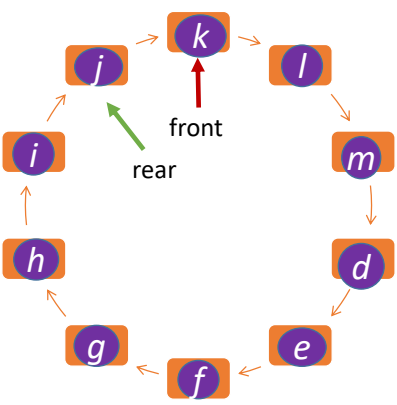
14

其他出题方法

判空：
 $(Q.rear+1)\%MaxSize==Q.front$



判满：
 $(Q.rear+1)\%MaxSize==Q.front?$



方案一：牺牲一个存储单元

方案二：增加辅助变量

王道考研/CSKAOYAN.COM

15

知识回顾与重要考点

实现思想

- 用静态数组存放数据元素，设置队头/队尾(front/rear)指针
- 循环队列：用模运算（取余）将存储空间在逻辑上变为“环状”
- $Q.rear=(Q.rear+1)\%MaxSize;$

重要考点

- 如何 初始化、入队、出队
- 如何 判空、判满
- 如何计算队列的长度

分析思路

- 确定front、rear指针的指向
 - ①rear指向队尾元素后一个位置
 - ②rear指向队尾元素
- 确定判空判满的方法
 - a. 牺牲一个存储单元
 - b. 增加 size 变量记录队列长度
 - c. 增加 tag = 0/1 用于标记最近的一次操作是 出队/入队
 - ...

思考：分别采用①a、①b、①c ②a、②b、②c 策略时，这些操作怎么实现？

王道考研/CSKAOYAN.COM

16

本节内容

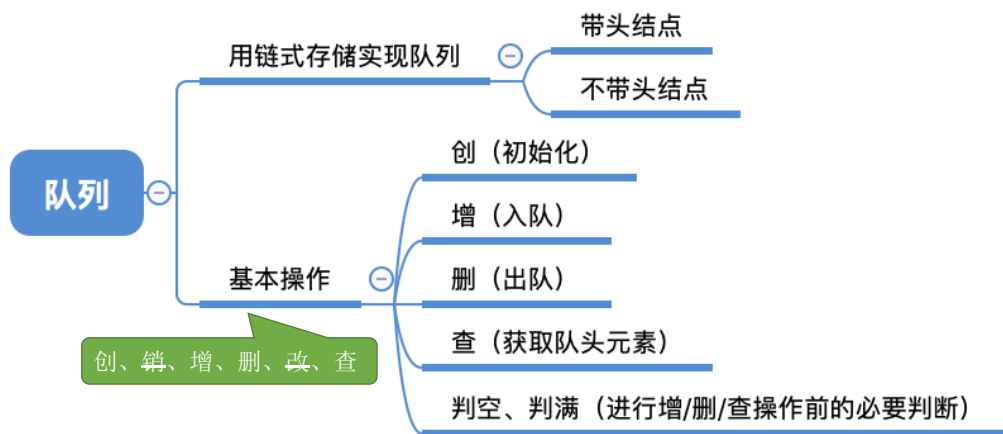
队列

链式实现

王道考研/CSKAOYAN.COM

1

知识总览



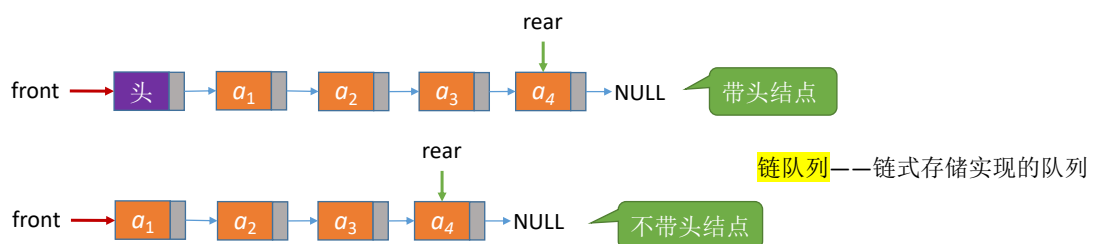
王道考研/CSKAOYAN.COM

2

队列的链式实现

```
typedef struct LinkNode{    //链式队列结点
    ElemType data;
    struct LinkNode *next;
}LinkNode;

typedef struct{            //链式队列
    LinkNode *front,*rear; //队列的队头和队尾指针
}LinkQueue;
```



王道考研/CSKAOYAN.COM

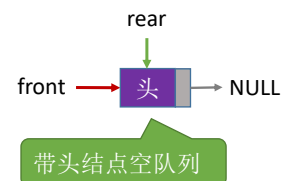
3

初始化（带头结点）

```
typedef struct LinkNode{    typedef struct{
    ElemType data;          LinkNode *front,*rear;
    struct LinkNode *next; }LinkQueue;
}LinkNode;

//初始化队列(带头结点)
void InitQueue(LinkQueue &Q){
    //初始时 front、rear 都指向头结点
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));
    Q.front->next=NULL;
}

void testLinkQueue(){
    LinkQueue Q;    //声明一个队列
    InitQueue(Q);  //初始化队列
    //...后续操作...
}
```



//判断队列是否为空

```
bool IsEmpty(LinkQueue Q){
    if(Q.front==Q.rear)
        return true;
    else
        return false;
}
```

王道考研/CSKAOYAN.COM

4

初始化（不带头结点）

```
//初始化队列(不带头结点)
void InitQueue(LinkQueue &Q){
    //初始时 front、rear 都指向NULL
```

```
    → Q.front=NULL;
    → Q.rear=NULL;
}
```

rear → NULL

front → NULL

不带头结点的空队列

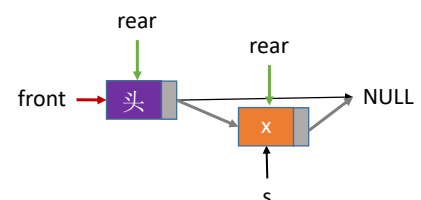
```
//判断队列是否为空（不带头结点）
bool IsEmpty(LinkQueue Q){
    if(Q.front==NULL)
        return true;
    else
        return false;
}
```

王道考研/CSKAOYAN.COM

5

入队（带头结点）

```
//新元素入队（带头结点）
void EnQueue(LinkQueue &Q, ElemType x){
    → LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));
    → s->data=x;
    → s->next=NULL;
    → Q.rear->next=s;    //新结点插入到rear之后
    → Q.rear=s;          //修改表尾指针
}
```



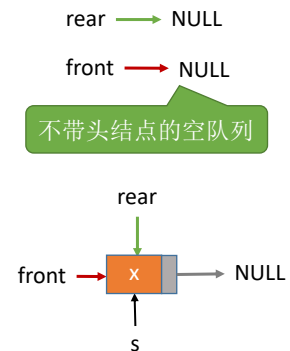
王道考研/CSKAOYAN.COM

6

入队（不带头结点）

//新元素入队（不带头结点）

```
void EnQueue(LinkQueue &Q, ElemType x){
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=x;
    s->next=NULL;
    if (Q.front == NULL){ //在空队列中插入第一个元素
        Q.front = s;      //修改队头队尾指针
        Q.rear=s;         //不带头结点的队列，第一个元素入队时需要特别处理
    } else {
        Q.rear->next=s;    //新结点插入到 rear 结点之后
        Q.rear=s;         //修改 rear 指针
    }
}
```



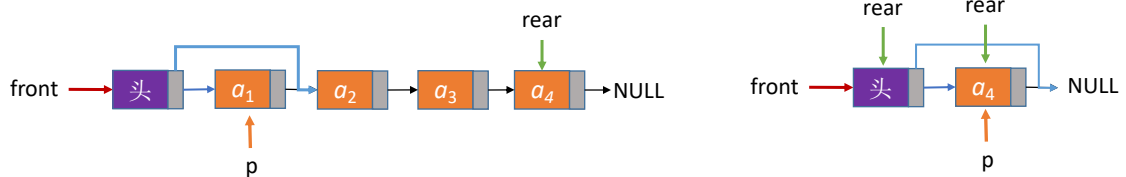
王道考研/CSKAOYAN.COM

7

出队（带头结点）

//队头元素出队（不带头结点）

```
bool DeQueue(LinkQueue &Q, ElemType &x){
    if(Q.front==Q.rear)
        return false; //空队
    LinkNode *p=Q.front->next;
    x=p->data;          //用变量x返回队头元素
    Q.front->next=p->next; //修改头结点的 next 指针
    if(Q.rear==p)       //此次是最后一个结点出队
        Q.rear=Q.front; //修改 rear 指针
    free(p);            //释放结点空间
    return true;
}
```



王道考研/CSKAOYAN.COM

8

出队（不带头结点）

//队头元素出队（不带头结点）

```
bool DeQueue(LinkQueue &Q, ElemType &x){
```

```
    if(Q.front==NULL)
```

```
        return false;
```

```
    LinkNode *p=Q.front;
```

```
    x=p->data;
```

```
    Q.front=p->next;
```

```
    if(Q.rear==p){
```

```
        Q.front = NULL;
```

```
        Q.rear = NULL;
```

```
    }
```

```
    free(p);
```

```
    return true;
```

```
}
```

//空队

//p指向此次出队的结点

//用变量x返回队头元素

//修改 front 指针

//此次是最后一个结点出队

//front 指向 NULL

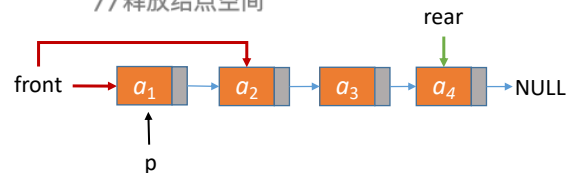
//rear 指向 NULL

//释放结点空间

rear → NULL

front → NULL

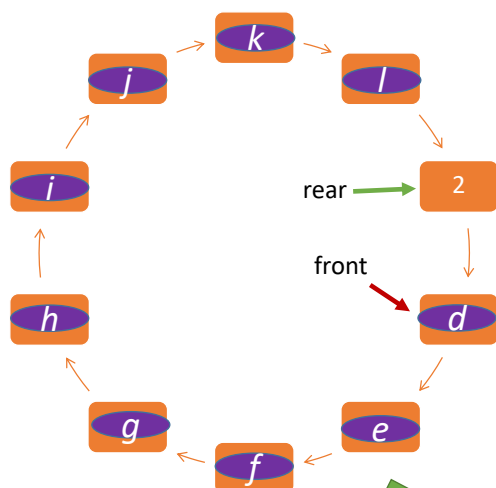
不带头结点的空队列



王道考研/CSKAOYAN.COM

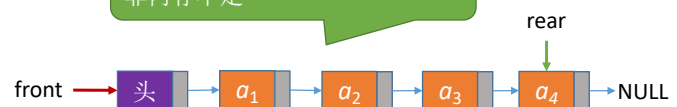
9

队列满的条件



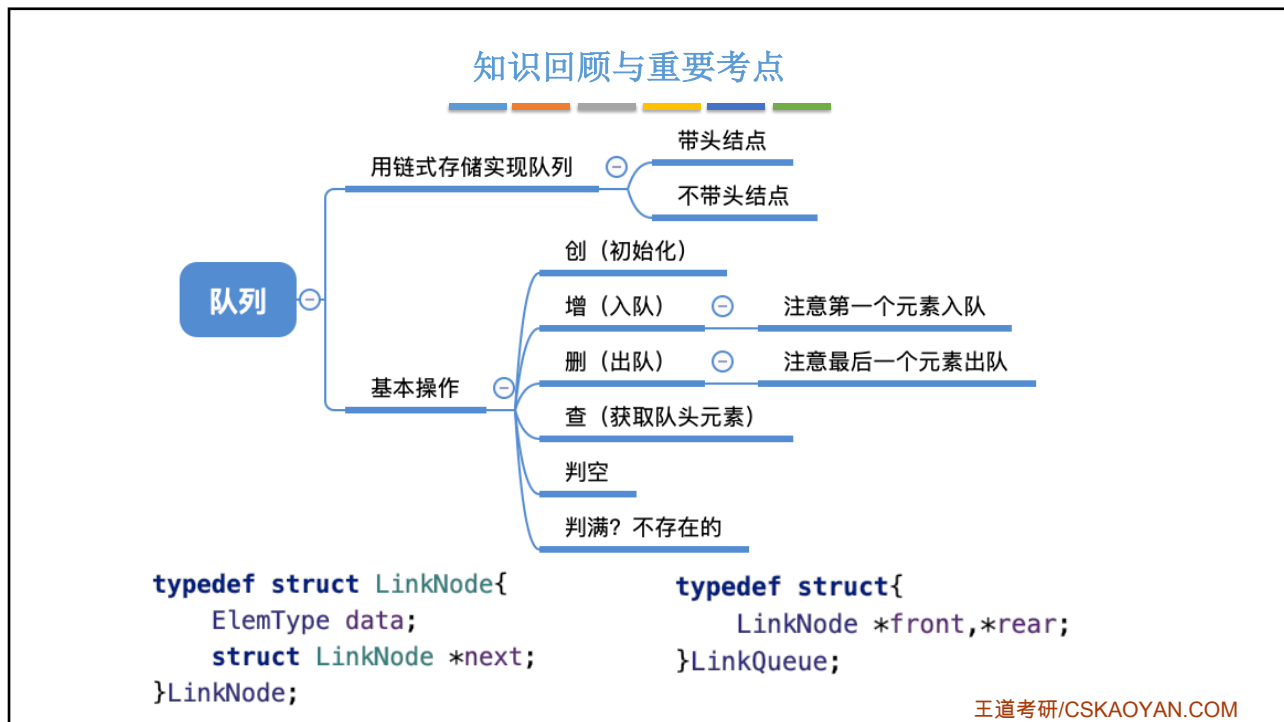
链式存储——一般不会队满，除非内存不足

顺序存储——预分配的空间耗尽时队满



王道考研/CSKAOYAN.COM

10



11

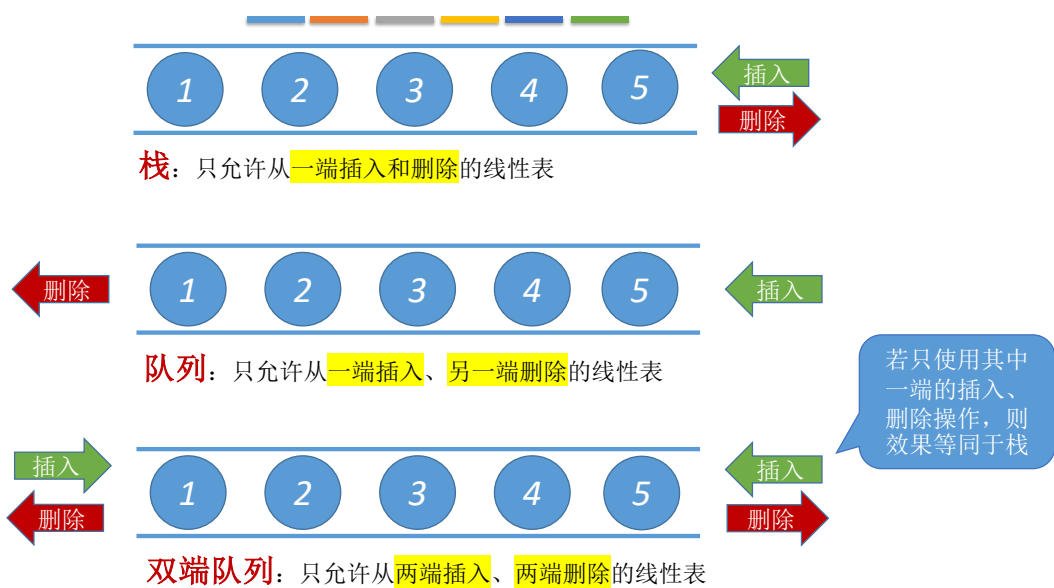
本节内容

双端队列

王道考研/CSKAOYAN.COM

1

双端队列



王道考研/CSKAOYAN.COM

2

双端队列

插入

删除

1

2

3

4

5

插入

删除

双端队列：只允许从两端插入、两端删除的线性表

删除

1

2

3

4

5

插入

删除

输入受限的双端队列：只允许从一端插入、两端删除的线性表

插入

1

2

3

4

5

插入

删除

输出受限的双端队列：只允许从两端插入、一端删除的线性表

王道考研/CSKAOYAN.COM

3

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$A_4^4 = 4! = 24$

1,2,3,4

2,1,3,4

3,1,2,4

4,1,2,3

1,2,4,3

2,1,4,3

3,1,4,2

4,1,3,2

1,3,2,4

2,3,1,4

3,2,1,4

4,2,1,3

1,3,4,2

2,3,4,1

3,2,4,1

4,2,3,1

1,4,2,3

2,4,1,3

3,4,1,2

4,3,1,2

1,4,3,2

2,4,3,1

3,4,2,1

4,3,2,1

1

2

3

4

插入

删除

栈

王道考研/CSKAOYAN.COM

4

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

1

2

3

4

插入

删除

栈

$A_4^4 = 4! = 24$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

王道考研/CSKAOYAN.COM

5

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

1

2

3

4

插入

删除

栈

$A_4^4 = 4! = 24$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

王道考研/CSKAOYAN.COM

6

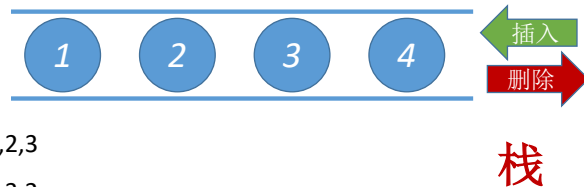
王道考研,cskaoyan.com

3

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

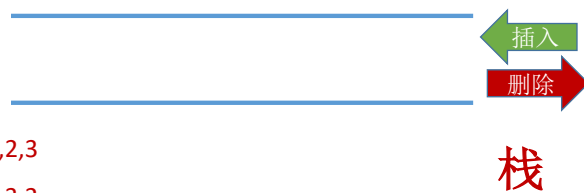
王道考研/CSKAOYAN.COM

7

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

卡特兰数：

14种合法
出栈序列

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{4+1} C_8^4 = 14$$

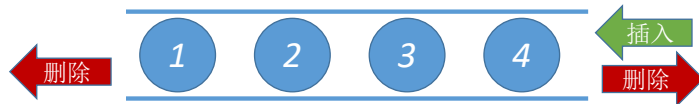
王道考研/CSKAOYAN.COM

8

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

栈中合法的序列，双端队列中一定也合法

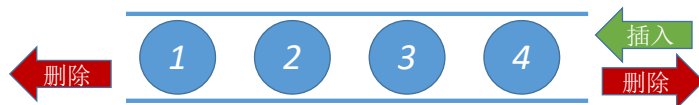
王道考研/CSKAOYAN.COM

9

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

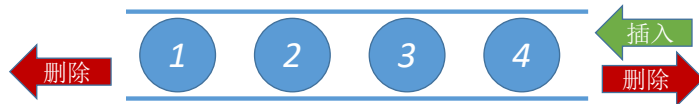
王道考研/CSKAOYAN.COM

10

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

王道考研/CSKAOYAN.COM

11

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	<u>4,1,3,2</u>
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

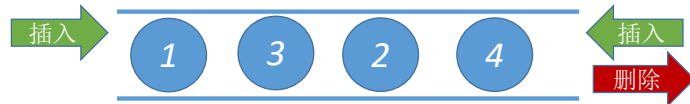
王道考研/CSKAOYAN.COM

12

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

栈中合法的序列，双端队列中一定也合法

王道考研/CSKAOYAN.COM

13

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

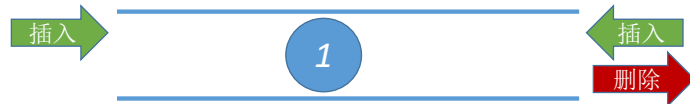
王道考研/CSKAOYAN.COM

14

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

王道考研/CSKAOYAN.COM

15

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



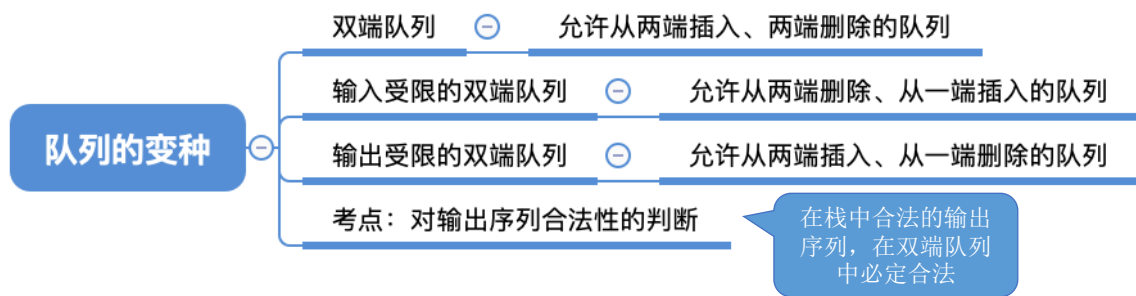
1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	<u>4,2,1,3</u>
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

王道考研/CSKAOYAN.COM

16

知识回顾与重要考点



回忆：栈的变种——共享栈

王道考研/CSKAOYAN.COM

本节内容

栈的应用

——括号匹配

王道考研/CSKAOYAN.COM

1

括号匹配问题

```
void test() {
    int a[10][10];
    int x = 10*(20*(1+1)-(3-2));
    printf("加油! 奥利给! ");
}
```

Expected ')' to match this '('

```
void test() {
    int a[10][10];
    int x = 10*(20*(1+1)-(3-2));
    printf("加油! 奥利给! ");
}
```

每一个 单身的人 得看透

想爱 就别怕伤痛

找一个 最爱的 深爱的

想爱的 亲爱的人 来告别单身

又常常羡慕
别人成双入对

王道考研/CSKAOYAN.COM

2

括号匹配问题

(((())))

① ② ③ ④ ④ ③ ② ①

最后出现的左括号最先被匹配 (LIFO)

可用“栈”实现该特性

((()) ())

① ② ③ ③ ② ④ ④ ①

每出现一个右括号，就“消耗”一个左括号

出栈

王道考研/CSKAOYAN.COM

3

算法演示

栈底

{ [(

{ (()) [] }

① ② ③ ③ ② ④ ④ ①

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

所有括号都能两两配对

王道考研/CSKAOYAN.COM

4

算法演示

栈底
↓

{	(	(			
---	---	---	--	--	--

{ (()] [] }

① ② ③ ③ ②

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

当前扫描到的右括号与栈顶左括号不匹配

王道考研/CSKAOYAN.COM

5

算法演示

栈底
↓

{	(	(			
---	---	---	--	--	--

{ (()) }] ()

① ② ③ ③ ② ①

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

扫描到右括号且栈空，——右括号单身

王道考研/CSKAOYAN.COM

6

算法演示

栈底

{ { [(

处理完所有括号后，栈非空——左括号单身

{ { (()) [] }

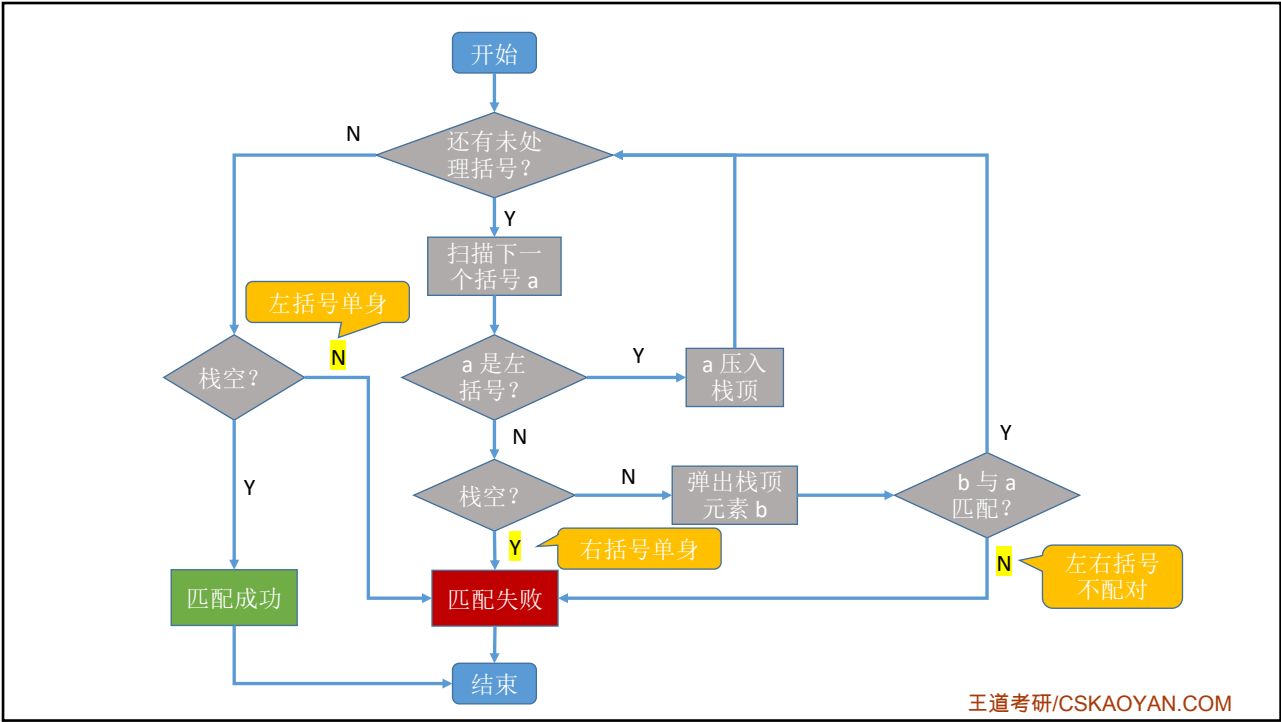
① ② ③ ④ ④ ③ ⑤ ⑤ ②

遇到左括号就入栈

遇到右括号，就“消耗”一个左括号

王道考研/CSKAOYAN.COM

7



8

算法实现

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S); //初始化一个栈
    for (int i=0; i<length; i++){
        if (str[i]=='(' || str[i]=='[' || str[i]=='{'){
            Push(S, str[i]); //扫描到左括号, 入栈
        } else {
            if (StackEmpty(S)) //扫描到右括号, 且当前栈空
                return false; //匹配失败

            char topElem;
            Pop(S, topElem); //栈顶元素出栈
            if (str[i]==')' && topElem!='(')
                return false;
            if (str[i]==']' && topElem!='[')
                return false;
            if (str[i]=='}' && topElem!='{')
                return false;
        }
    }
    return StackEmpty(S); //检索全部括号后, 栈空说明匹配成功
}
```

```
#define MaxSize 10 //定义栈中元素的最大个数
typedef struct{
    char data[MaxSize]; //静态数组存放栈中元素
    int top; //栈顶指针
} SqStack;
```

万一存满了可咋整?

可用链栈!

考试中可直接使用基本操作, 建议简要说明接口

```
//初始化栈
void InitStack(SqStack &S)
//判断栈是否为空
bool StackEmpty(SqStack S)
//新元素入栈
bool Push(SqStack &S, char x)
//栈顶元素出栈, 用x返回
bool Pop(SqStack &S, char &x)
```

练习: 不要使用基本操作, 动手实现完整代码

王道考研/CSKAOYAN.COM

9

知识回顾与重要考点

用栈实现**括号匹配**:

依次扫描所有字符, 遇到左括号入栈, 遇到右括号则弹出栈顶元素检查是否匹配。

匹配失败情况:

①左括号单身②右括号单身③左右括号不匹配

王道考研/CSKAOYAN.COM

10

本节内容

栈的应用
——表达式求值

王道考研/CSKAOYAN.COM

1

知识总览

表达式求值问题

三种算术表达式

中缀表达式

后缀表达式

前缀表达式

后缀表达式相关考点

中缀表达式转后缀表达式

后缀表达式求值

前缀表达式相关考点

中缀表达式转前缀表达式

前缀表达式求值

王道考研/CSKAOYAN.COM

2

大家熟悉的算数表达式



Reference: Wikipedia
—— Reverse Polish notation

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③ ② ① ④ ⑦ ⑥ ⑤

$$15 \div 7 - 1 + 1 \times 3 - 2 + 1 + 1$$

① ② ④ ③ ⑤ ⑥ ⑦

由三个部分组成：操作数、运算符、界限符

界限符是必不可少的，
反映了计算的先后顺序

王道考研/CSKAOYAN.COM

3

波兰数学家的灵感

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$



一个灵感：可以不用界限符也能无歧义地表达运算顺序

Reverse Polish notation (逆波兰表达式 = 后缀表达式)

Polish notation (波兰表达式 = 前缀表达式)

王道考研/CSKAOYAN.COM

4

中缀、后缀、前缀表达式

运算符在两个操作数中间

中缀表达式

$a + b$

$a + b - c$

$a + b - c * d$

规则：运算符在两个操作数后面

后缀表达式

$a b +$

$a b + c -$

$a b + c d * -$

规则：运算符在两个操作数前面

前缀表达式

$+ a b$

$- + a b c$

$- + a b * c d$

王道考研/CSKAOYAN.COM

5

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

③ ② ① ④ ⑦ ⑥ ⑤

中缀表达式

$15 \ 7 \ 1 \ 1 \ + \ - \ \div \ 3 \ \times \ 2 \ 1 \ 1 \ + \ + \ -$

① ② ③ ④ ⑤ ⑥ ⑦

后缀表达式

王道考研/CSKAOYAN.COM

6

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

运算顺序不唯一，因此对应的
后缀表达式也不唯一

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④



① ② ③ ④ ⑤

$ABCD - * + EF / -$

$A + B * (C - D) - E / F$

⑤ ③ ② ④ ①



② ③ ① ④ ⑤

$ABCD - * EF / - +$

私房菜：“左优先”原则，不要FreeStyle，保证手算和机算结果相同

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

客观来看两种都正确，只是“机算”结果是前者

王道考研/CSKAOYAN.COM

7

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

运算顺序不唯一，因此对应的
后缀表达式也不唯一

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

可保证运算顺序唯一

$A + B - C * D / E + F$

① ④ ② ③ ⑤

$AB + CD * E / - F +$

① ② ③ ④ ⑤

王道考研/CSKAOYAN.COM

8

后缀表达式的计算（手算）

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

③ ② ① ④ ⑦ ⑥ ⑤

中缀表达式

15 7 1 1 + - ÷ 3 × 2 1 1 + + -

① ② ③ ④ ⑤ ⑥ ⑦

后缀表达式

后缀表达式的手算方法：
从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，
合体为一个操作数

注意：两个操作数的左右顺序

后缀表达式的计算（手算）

后缀表达式的手算方法：
从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，
合体为一个操作数

注意：两个操作数的左右顺序

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④

① ② ③ ④ ⑤

A B C D - * + E F / -

后缀表达式的计算（手算）

后缀表达式的手算方法：
从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，
合体为一个操作数

注意：两个操作数的左右顺序

$A + B - C * D / E + F$
① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $AB + CD * E / - F +$



认真思考

特点：最后出现的
操作数先被运算



耶！

LIFO（后进先出）

栈！！

王道考研/CSKAOYAN.COM

11

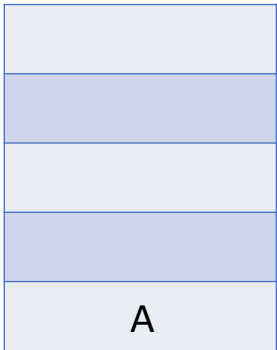
后缀表达式的计算（机算）

用栈实现后缀表达式的计算：
①从左往右扫描下一个元素，直到处理完所有元素
②若扫描到操作数则压入栈，并回到①；否则执行③
③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

$A + B - C * D / E + F$
① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $AB + CD * E / - F +$

栈



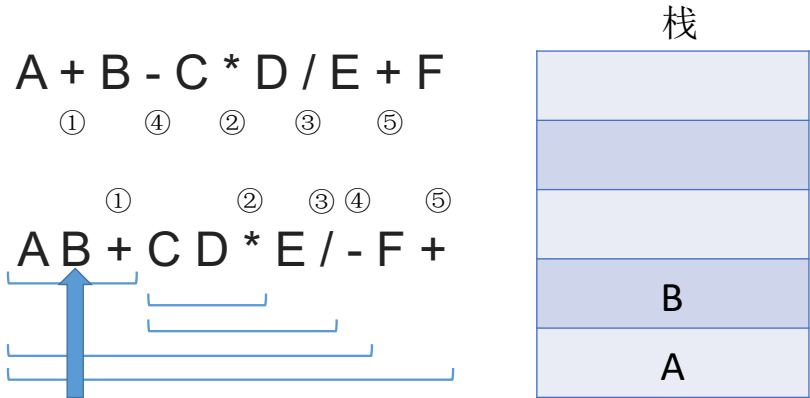
王道考研/CSKAOYAN.COM

12

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



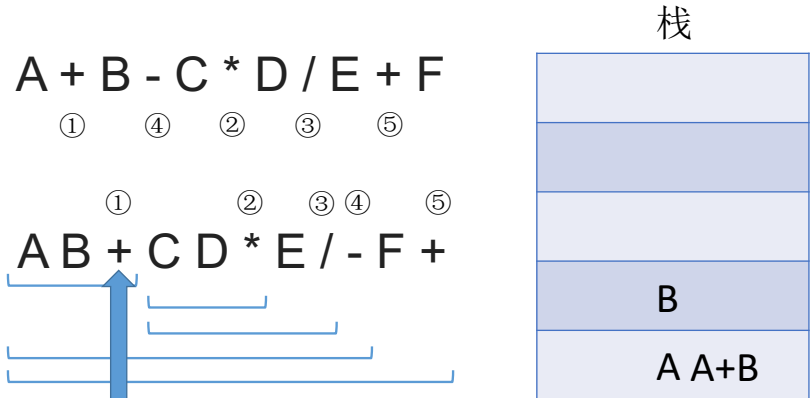
王道考研/CSKAOYAN.COM

13

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

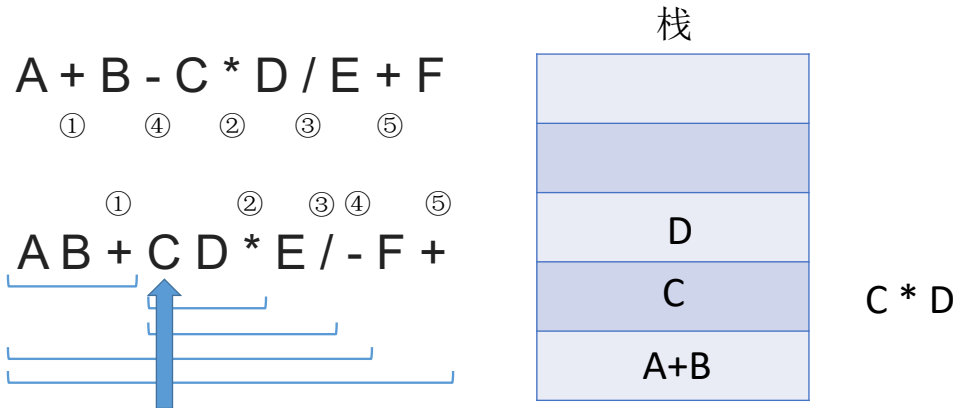


王道考研/CSKAOYAN.COM

14

后缀表达式的计算（机算）

- 用栈实现后缀表达式的计算：
- ①从左往右扫描下一个元素，直到处理完所有元素
 - ②若扫描到操作数则压入栈，并回到①；否则执行③
 - ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

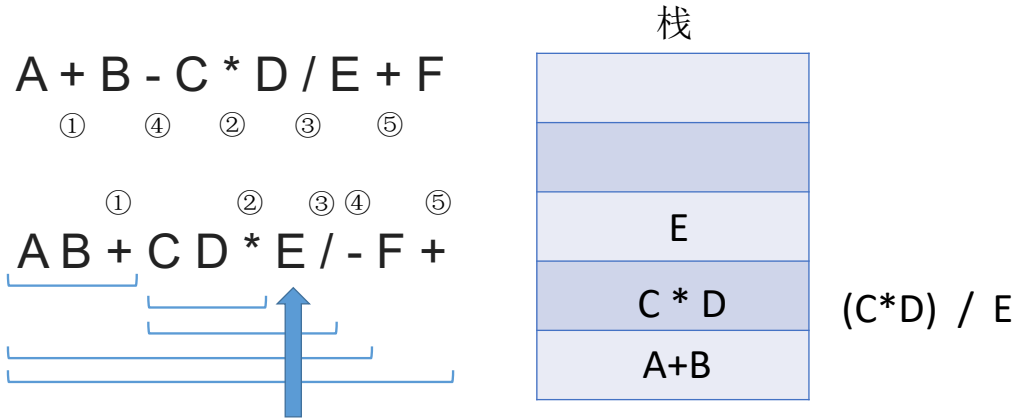


王道考研/CSKAOYAN.COM

15

后缀表达式的计算（机算）

- 用栈实现后缀表达式的计算：
- ①从左往右扫描下一个元素，直到处理完所有元素
 - ②若扫描到操作数则压入栈，并回到①；否则执行③
 - ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

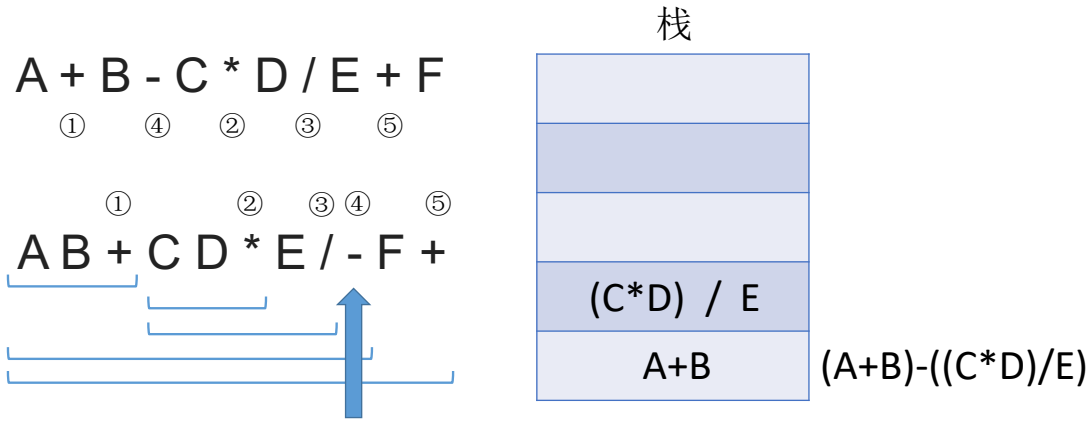


王道考研/CSKAOYAN.COM

16

后缀表达式的计算（机算）

- 用栈实现后缀表达式的计算：
- ①从左往右扫描下一个元素，直到处理完所有元素
 - ②若扫描到操作数则压入栈，并回到①；否则执行③
 - ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

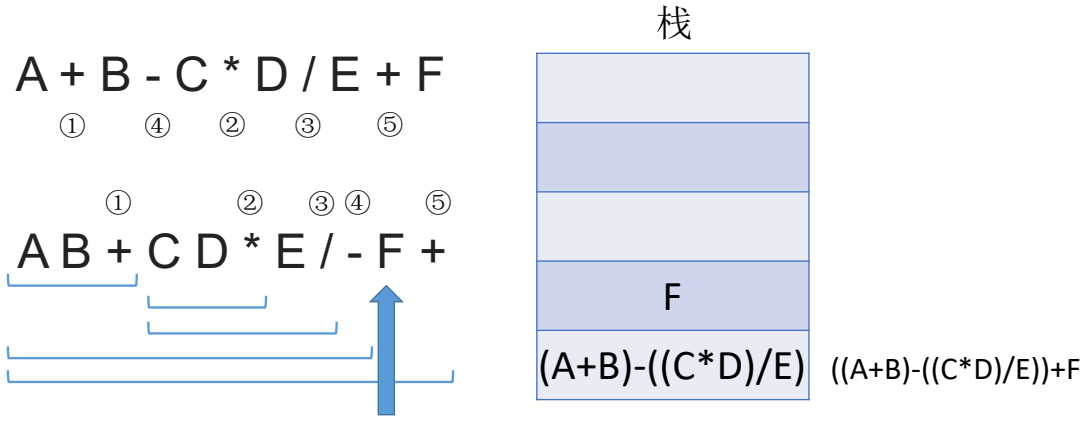


王道考研/CSKAOYAN.COM

17

后缀表达式的计算（机算）

- 用栈实现后缀表达式的计算：
- ①从左往右扫描下一个元素，直到处理完所有元素
 - ②若扫描到操作数则压入栈，并回到①；否则执行③
 - ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



王道考研/CSKAOYAN.COM

18

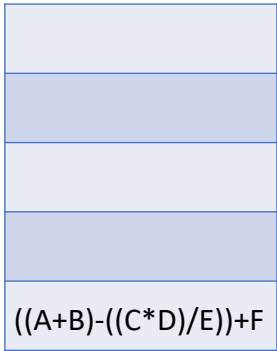
后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

栈

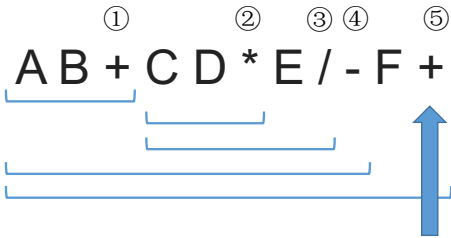


A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤

AB + CD * E / - F +



若表达式合法，则最后栈中只会留下一个元素，就是最终结果

王道考研/CSKAOYAN.COM

19

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

后缀表达式适用于基于栈的编程语言（stack-oriented programming language），如：Forth、PostScript

栈



((15 ÷ (7 - (1 + 1))) × 3) - (2 + (1 + 1))

③ ② ① ④ ⑦ ⑥ ⑤

15 7 1 1 + - ÷ 3 × 2 1 1 + + -

思考：后缀表达式怎么转中缀？

放弃思考



王道考研/CSKAOYAN.COM

20

中缀表达式转前缀表达式（手算）

中缀转前缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「运算符 左操作数 右操作数」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

“右优先”原则：只要右边的运算符能先计算，就优先算右边的

$$A + B * (C - D) - E / F$$

③ ② ① ⑤ ④ ⑤ ③ ② ④ ①

⑤ ③ ② ① ④ ⑤ ④ ③ ② ①

$$- + A * B - C D / E F$$

$$+ A - * B - C D / E F$$

中缀表达式转前缀表达式（手算）

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③ ② ① ④ ⑦ ⑥ ⑤

中缀转后缀：
“左优先”

$$15 \ 7 \ 1 \ 1 \ + \ - \ \div \ 3 \ \times \ 2 \ 1 \ 1 \ + \ + \ -$$

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

⑥ ④ ③ ⑤ ⑦ ② ①

中缀转前缀：
“右优先”

$$- \ \div \ \times \ 15 \ - \ 7 \ + \ 1 \ 1 \ 3 \ + \ 2 \ + \ 1 \ 1$$

⑦ ⑥ ⑤ ④ ③ ② ①

前缀表达式的计算

用栈实现前缀表达式的计算：

- ①从右往左扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“左操作数”

栈

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

⑥ ④ ③ ⑤ ⑦ ② ①

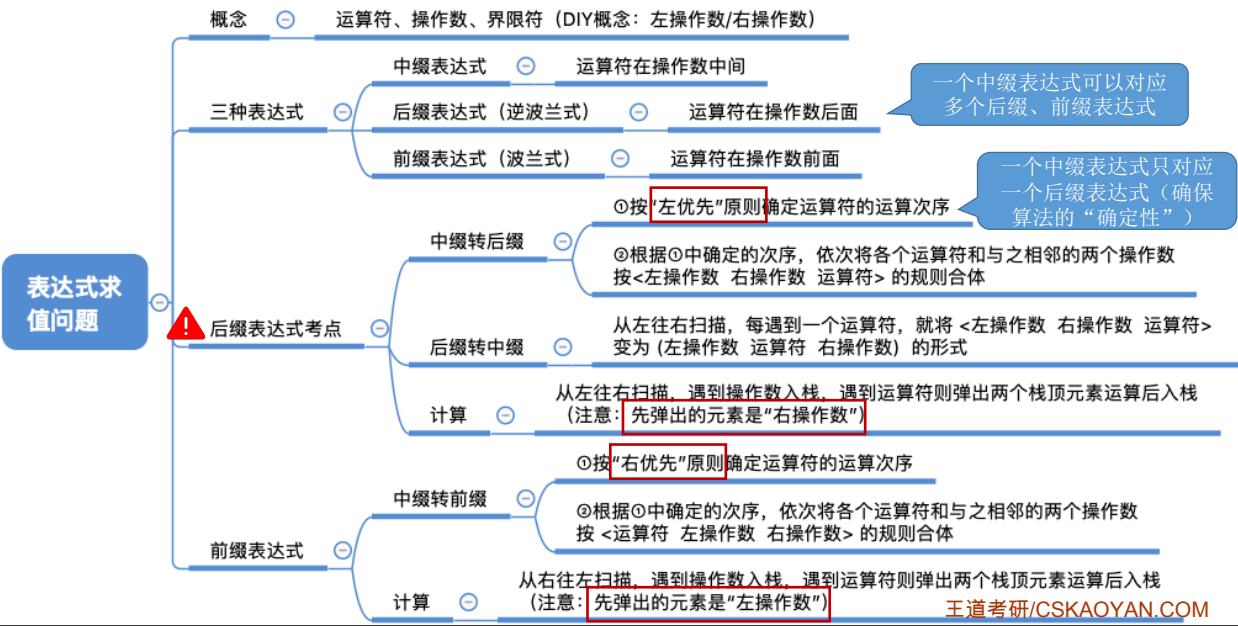
$- \div \times 15 - 7 + 1 \quad 1 \quad 3 + 2 + 1 \quad 1$

⑦ ⑥ ⑤ ④ ③ ② ①



王道考研/CSKAOYAN.COM

知识回顾与重要考点



本节内容

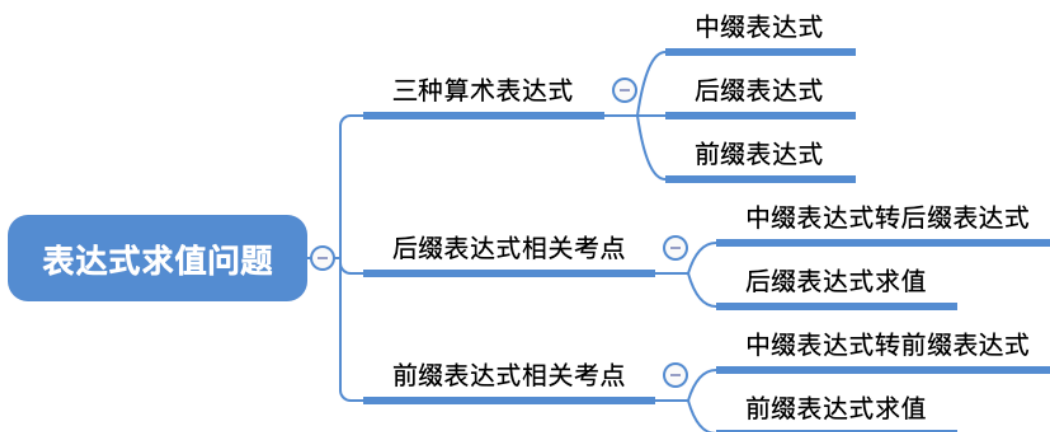
栈的应用

——表达式求值

王道考研/CSKAOYAN.COM

1

知识总览



王道考研/CSKAOYAN.COM

2

知识总览

表达式求值问题（第二季）

中缀表达式转后缀表达式（机算，用栈实现）

中缀表达式的计算（用栈实现）

王道考研/CSKAOYAN.COM

3

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：
① 确定中缀表达式中各个运算符的运算顺序
② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
③ 如果还有运算符没被处理，就继续 ②
“左优先”原则：只要左边的运算符能先计算，就优先算左边的

$A + B - C * D / E + F$
① ④ ② ③ ⑤

$AB + CD * E / - F +$
① ② ③ ④ ⑤

王道考研/CSKAOYAN.COM

4

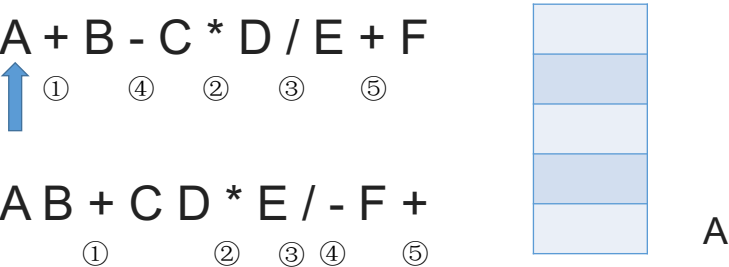
中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

*/ 优先级高于 + -

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

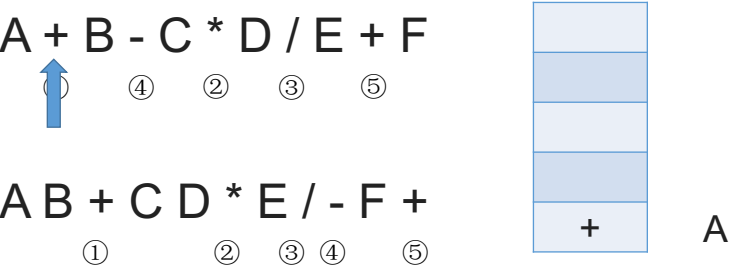
5

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

6

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

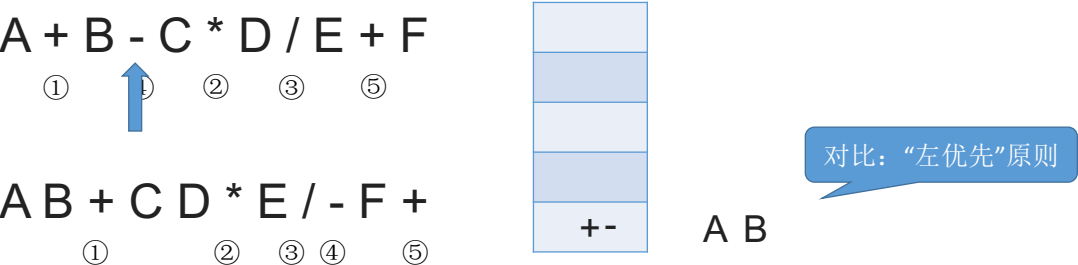
7

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

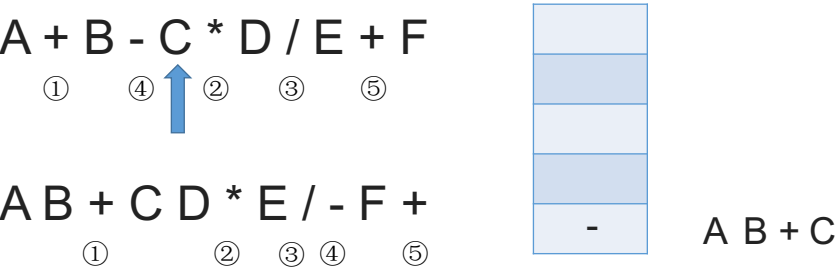
8

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

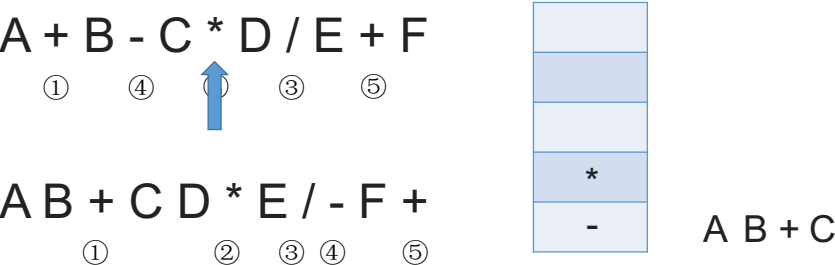
9

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

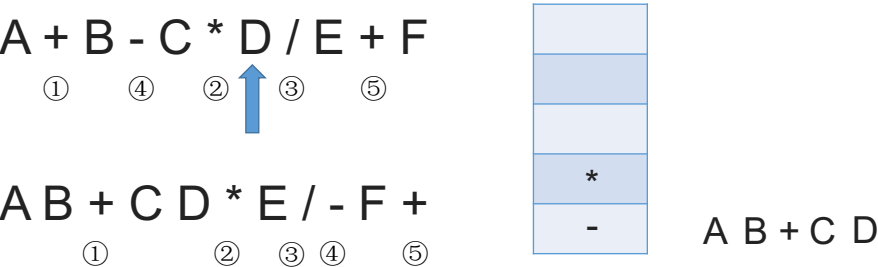
10

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

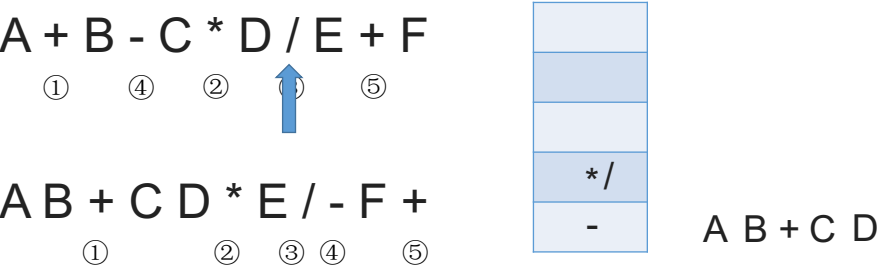
11

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

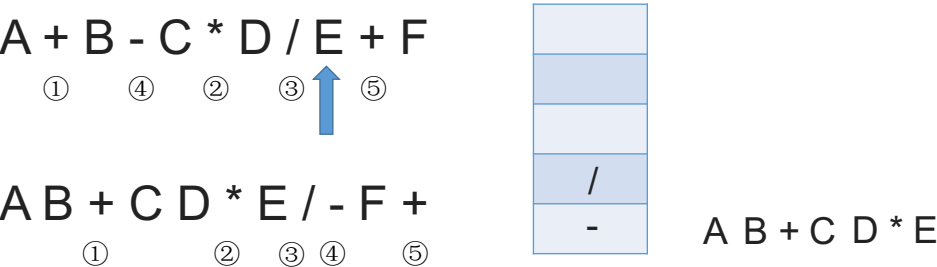
12

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

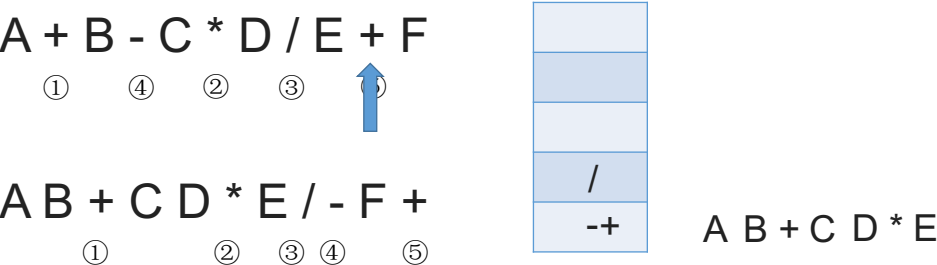
13

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

14

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

① ④ ② ③ ⑤

↑

A B + C D * E / - F +

① ② ③ ④ ⑤

+

A B + C D * E / - F

王道考研/CSKAOYAN.COM

15

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

① ④ ② ③ ⑤

↑

A B + C D * E / - F +

① ② ③ ④ ⑤

奈斯

A B + C D * E / - F +

王道考研/CSKAOYAN.COM

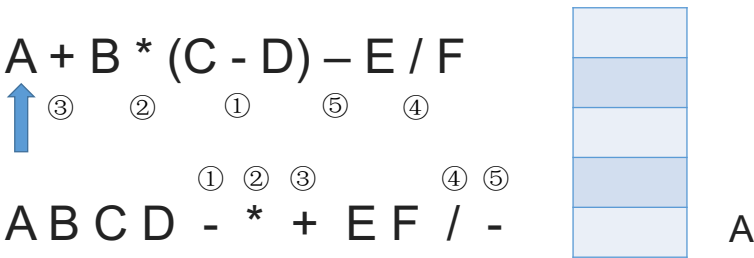
16

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

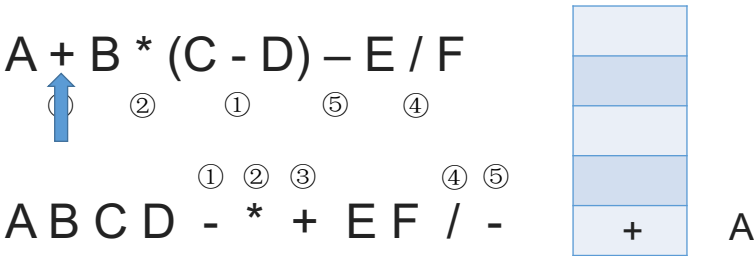
17

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

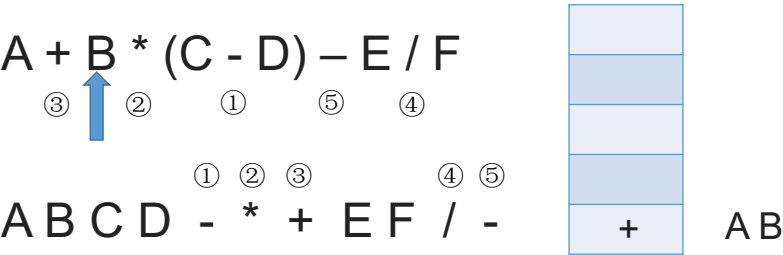
18

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

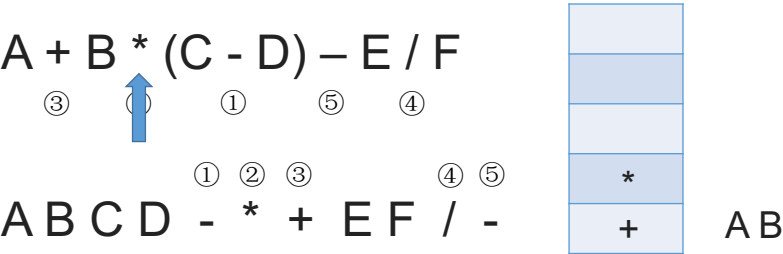
19

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

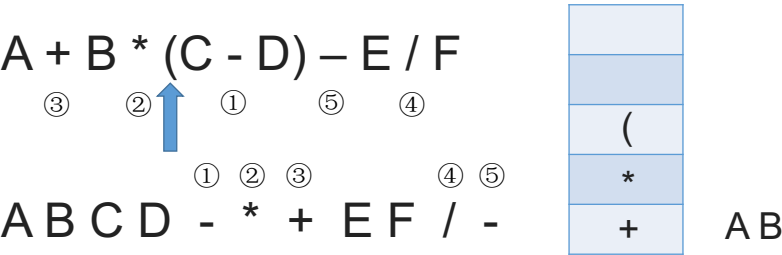
20

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

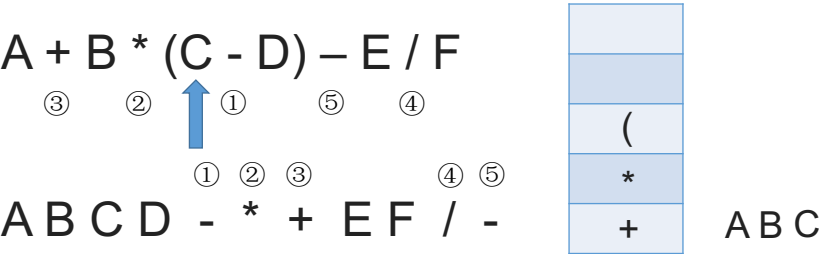
21

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

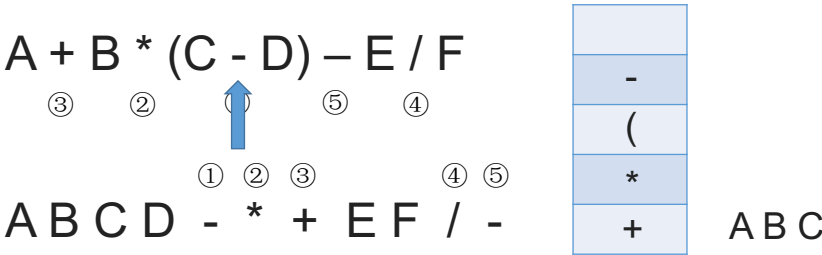
22

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

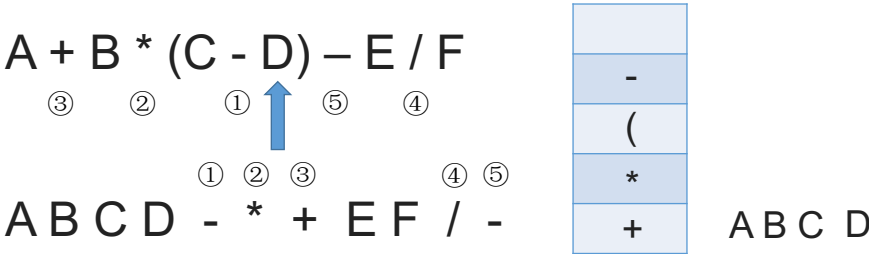
23

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

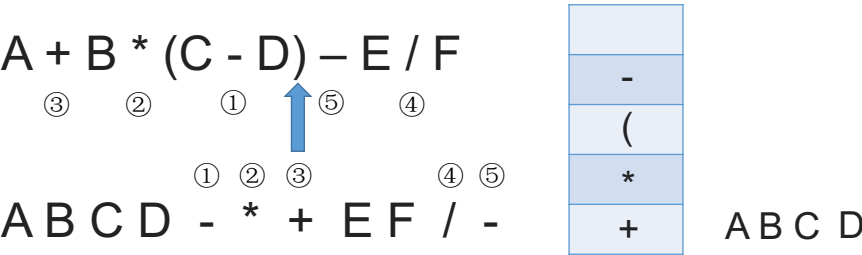
24

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

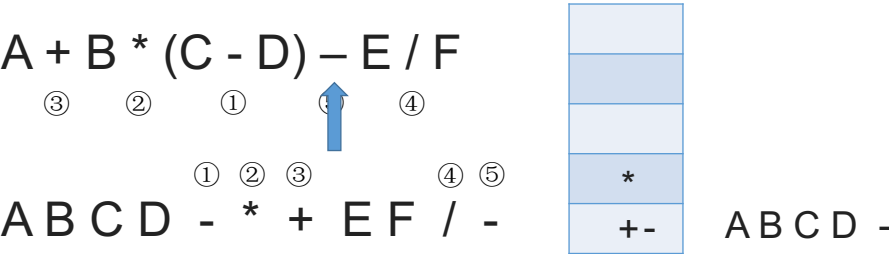
25

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

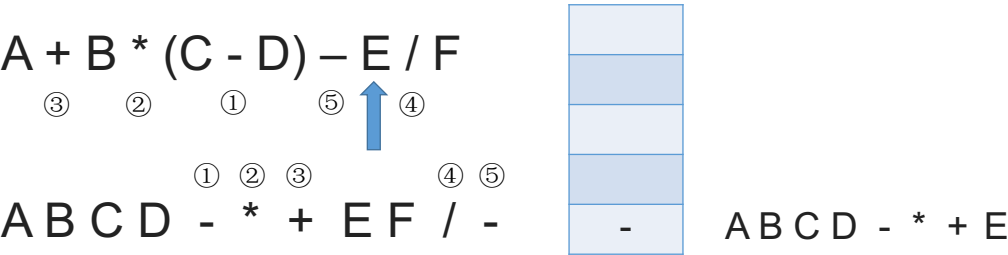
26

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

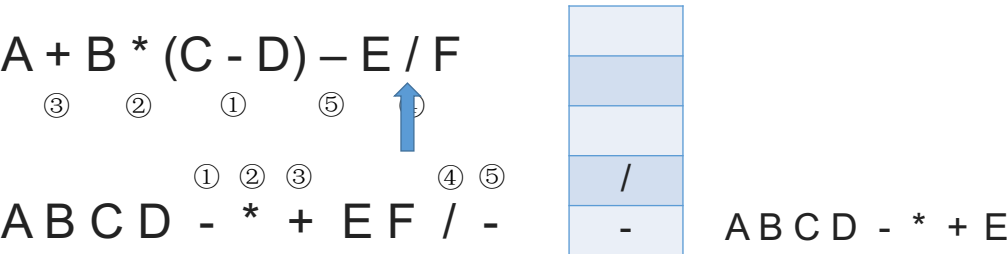
27

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

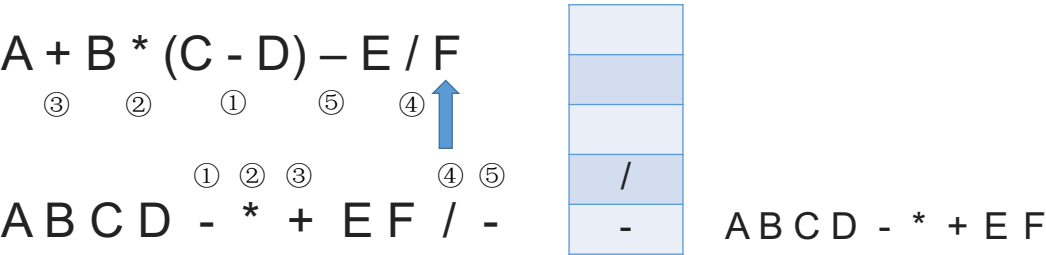
28

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

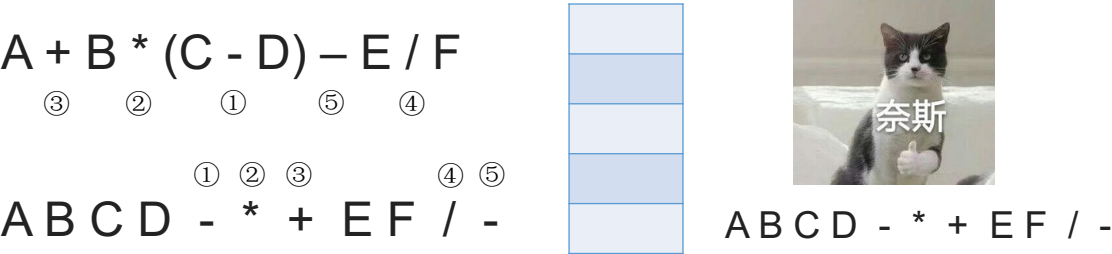
29

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



王道考研/CSKAOYAN.COM

30

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

③ ② ① ④ ⑦ ⑥ ⑤

15 7 1 1 + - ÷ 3 × 2 1 1 + + -



王道考研/CSKAOYAN.COM

31

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

中缀转后缀
+
后缀表达式求值
两个算法的结合

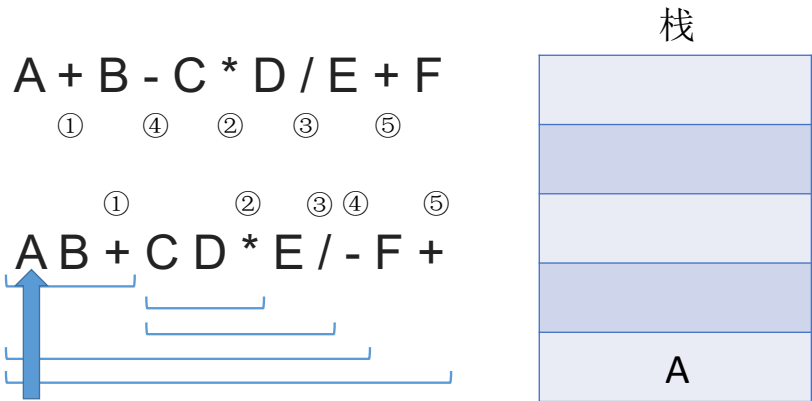
王道考研/CSKAOYAN.COM

32

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



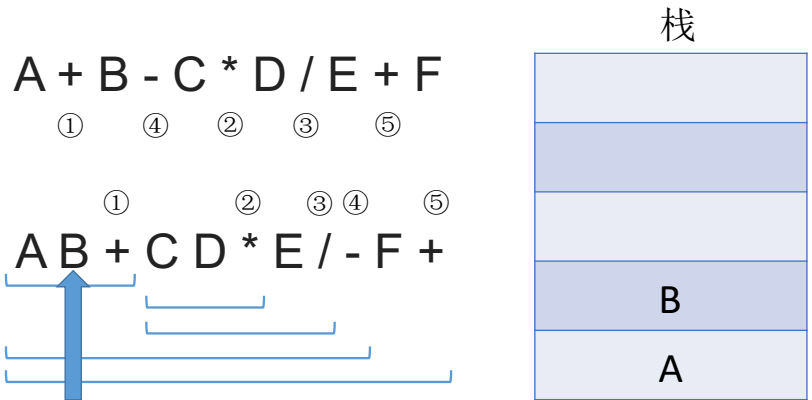
王道考研/CSKAOYAN.COM

33

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



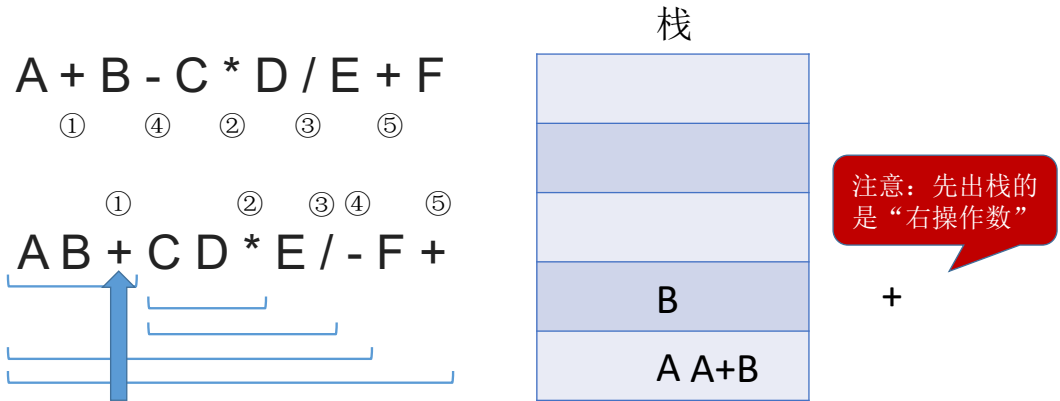
王道考研/CSKAOYAN.COM

34

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



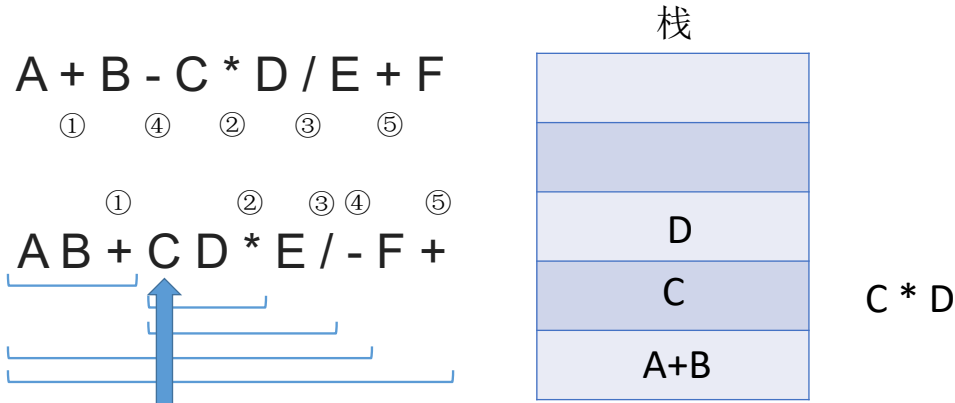
王道考研/CSKAOYAN.COM

35

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



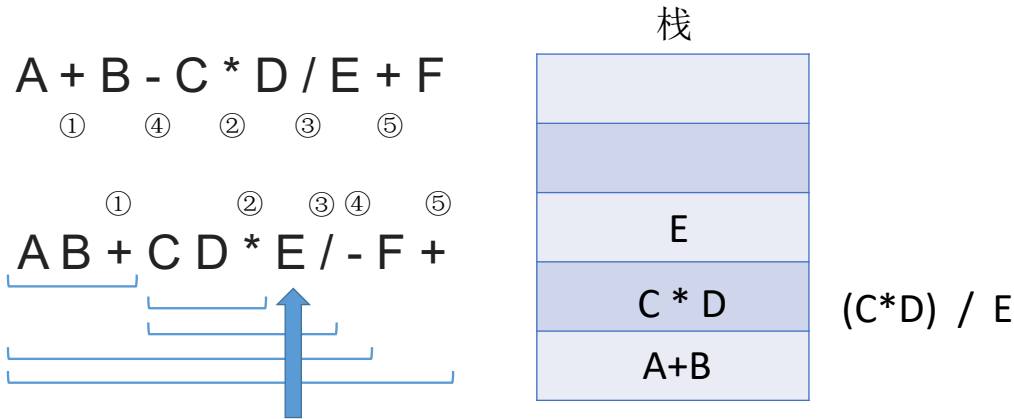
王道考研/CSKAOYAN.COM

36

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



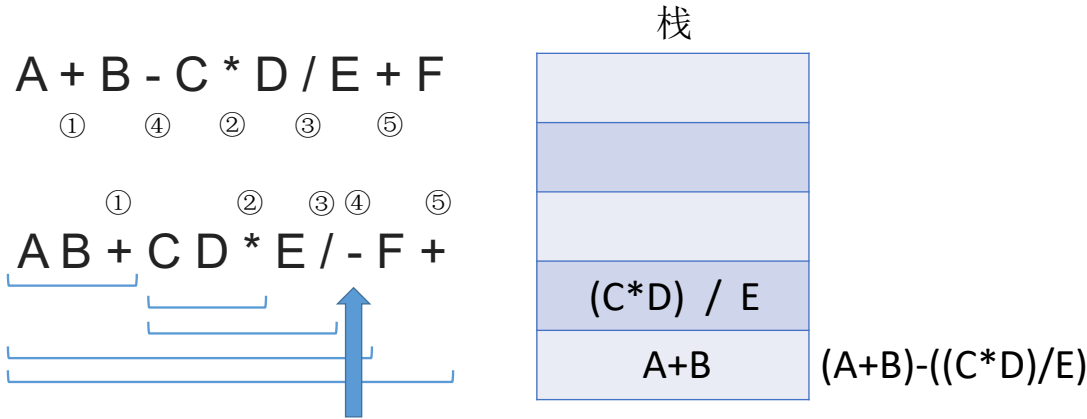
王道考研/CSKAOYAN.COM

37

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



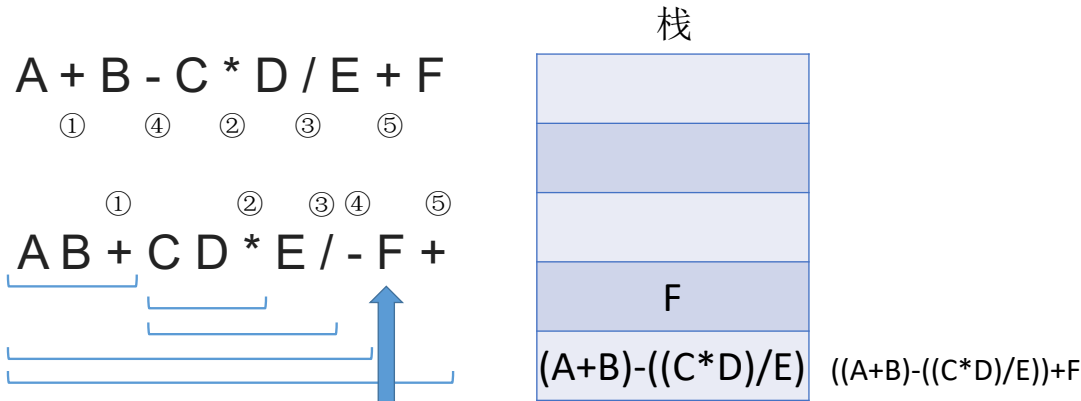
王道考研/CSKAOYAN.COM

38

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①



王道考研/CSKAOYAN.COM

39

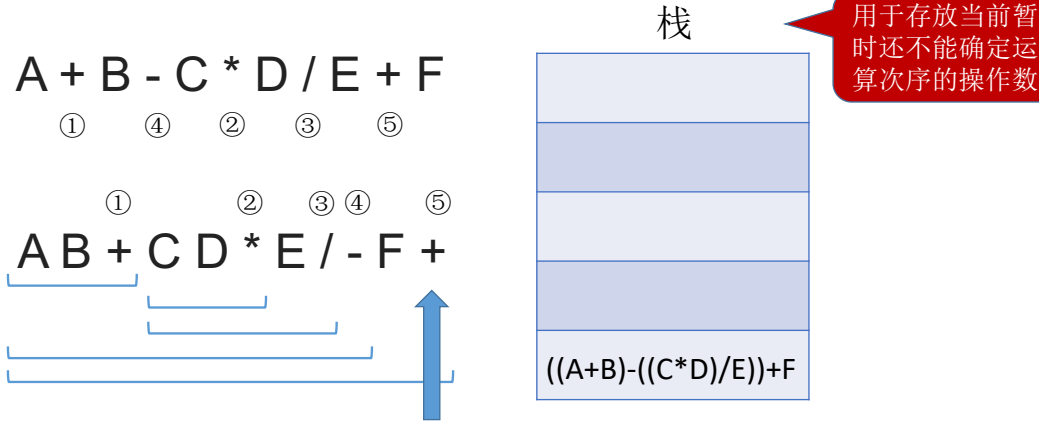
后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

用于存放当前暂时还不能确定运算次序的操作数



王道考研/CSKAOYAN.COM

40

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

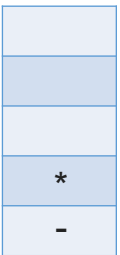
A + B - C * D / E + F

① ④ ② ③ ⑤



A B + C D * E / - F +

① ② ③ ④ ⑤



用于存放当前暂时还不能确定运算次序的运算符

A B + C D

王道考研/CSKAOYAN.COM

41

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

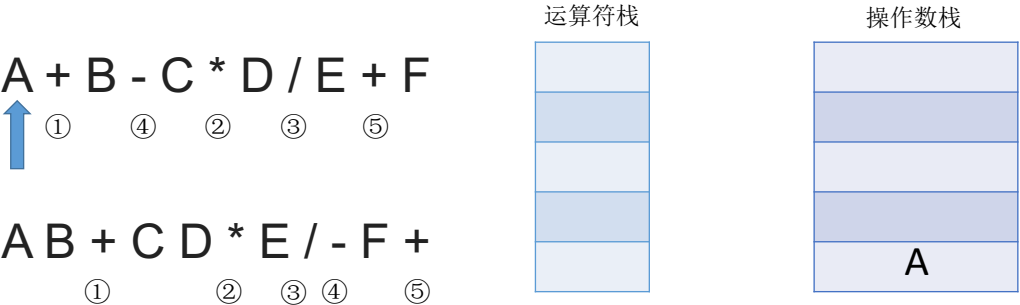
中缀转后缀
+
后缀表达式求值
两个算法的结合

王道考研/CSKAOYAN.COM

42

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

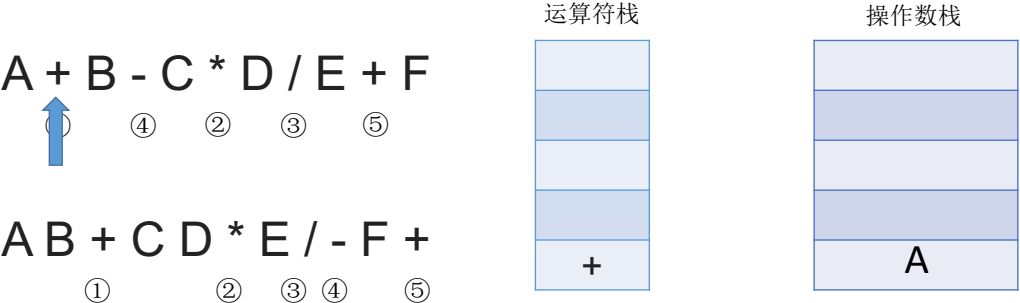


王道考研/CSKAOYAN.COM

43

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

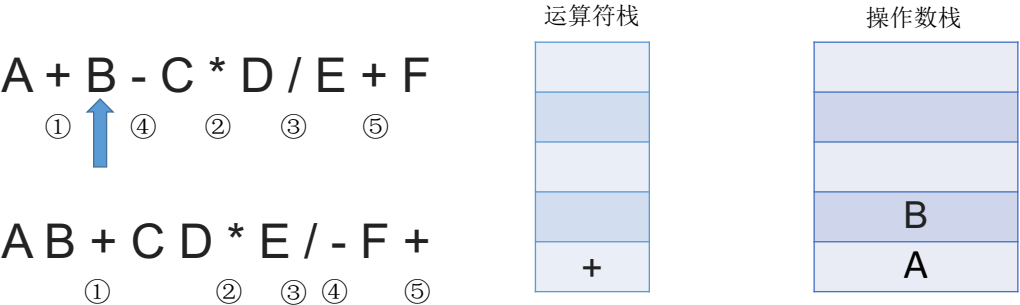


王道考研/CSKAOYAN.COM

44

中缀表达式的计算（用栈实现）

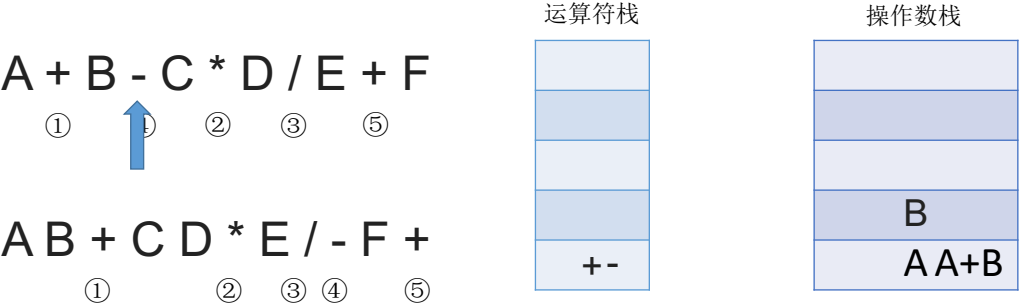
用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



45

中缀表达式的计算（用栈实现）

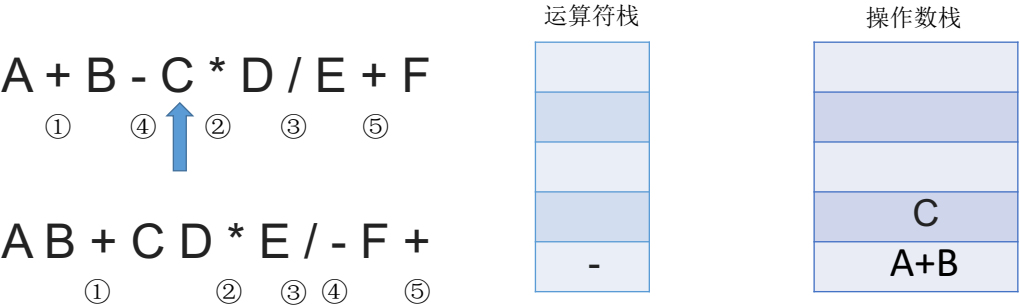
用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



46

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

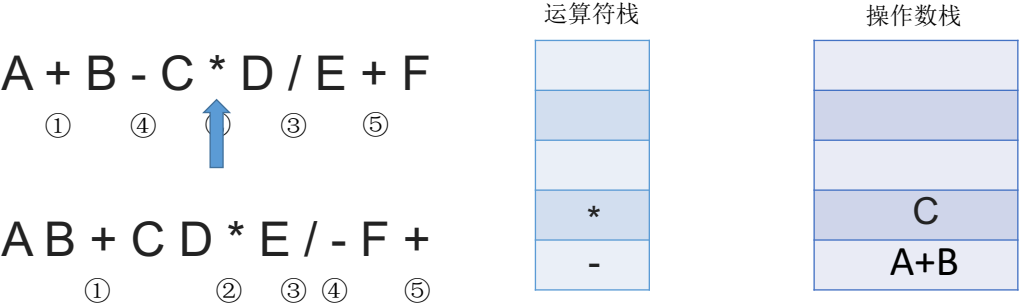


王道考研/CSKAOYAN.COM

47

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

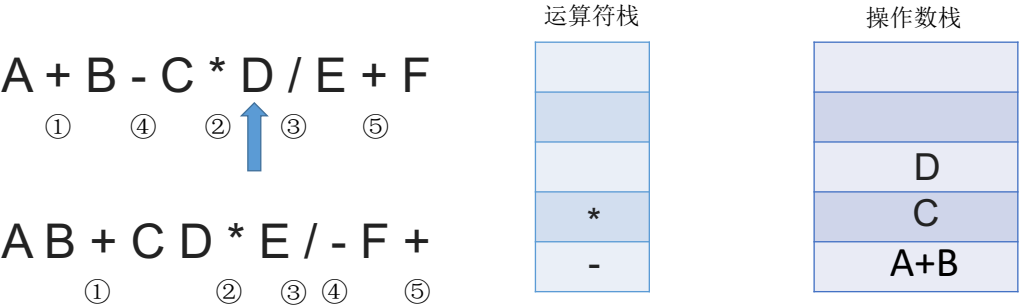


王道考研/CSKAOYAN.COM

48

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

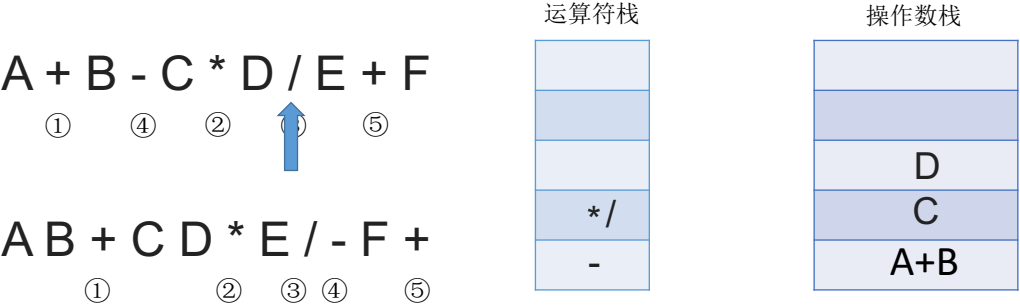


王道考研/CSKAOYAN.COM

49

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

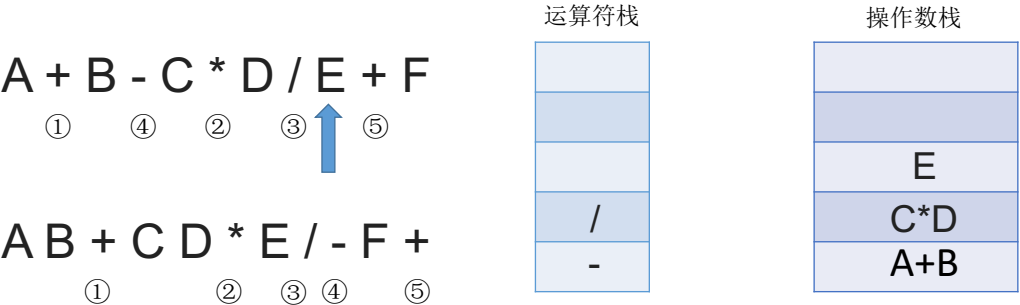


王道考研/CSKAOYAN.COM

50

中缀表达式的计算（用栈实现）

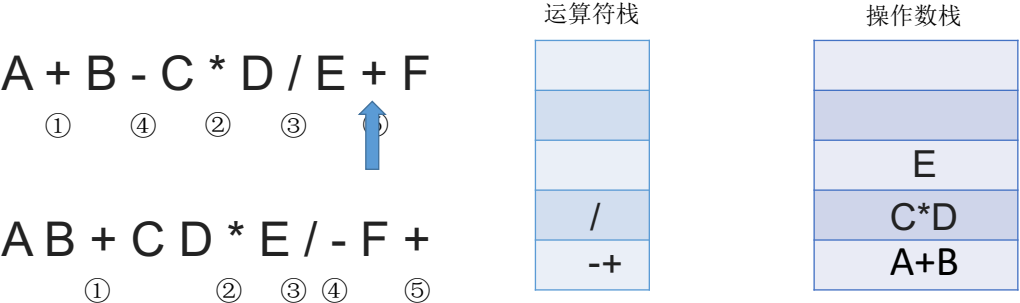
用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



51

中缀表达式的计算（用栈实现）

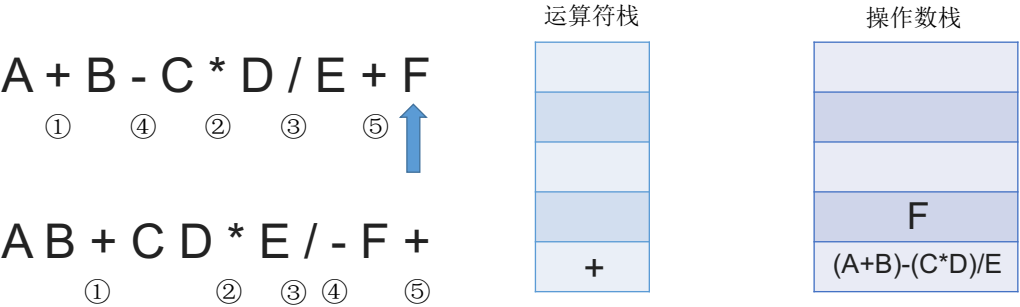
用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



52

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

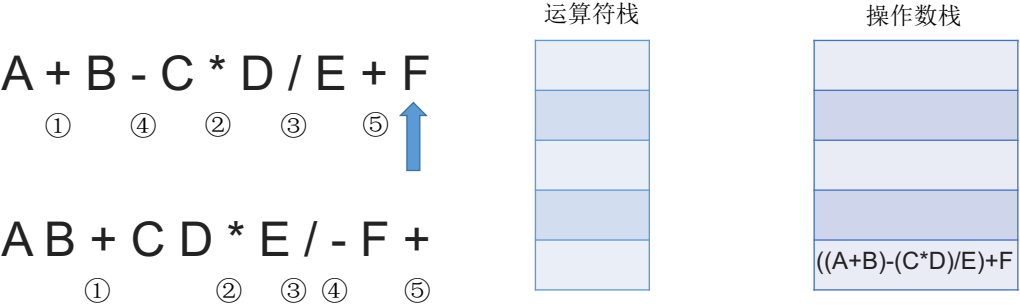


王道考研/CSKAOYAN.COM

53

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



王道考研/CSKAOYAN.COM

54

知识回顾与重要考点

用栈实现中缀表达式转后缀表达式：

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到“)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

用栈实现后缀表达式的计算：

- ① 从左往右扫描下一个元素，直到处理完所有元素
- ② 若扫描到操作数则压入栈，并回到①；否则执行③
- ③ 若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

用栈实现中缀表达式的计算：

初始化两个栈，**操作数栈**和**运算符栈**

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

王道考研/CSKAOYAN.COM

本节内容

栈的应用
——递归

王道考研/CSKAOYAN.COM

1

函数调用背后的过程

```
void main() {  
    int a, b, c;  
    ...  
    func1 (a, b);  
#1 → c=a+b;  
    ...  
}
```

```
void func1 (int a, int b) {  
    int x;  
    ...  
    func2 (x);  
#2 → x=x+10086;  
    ...  
}
```

```
void func2 (int x) {  
    int m, n;  
    ...  
}
```

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用栈存储：

- ① 调用返回地址
- ② 实参
- ③ 局部变量

func2: #2, x, m, n

func1: #1, a, b, x

main: ... a, b, c

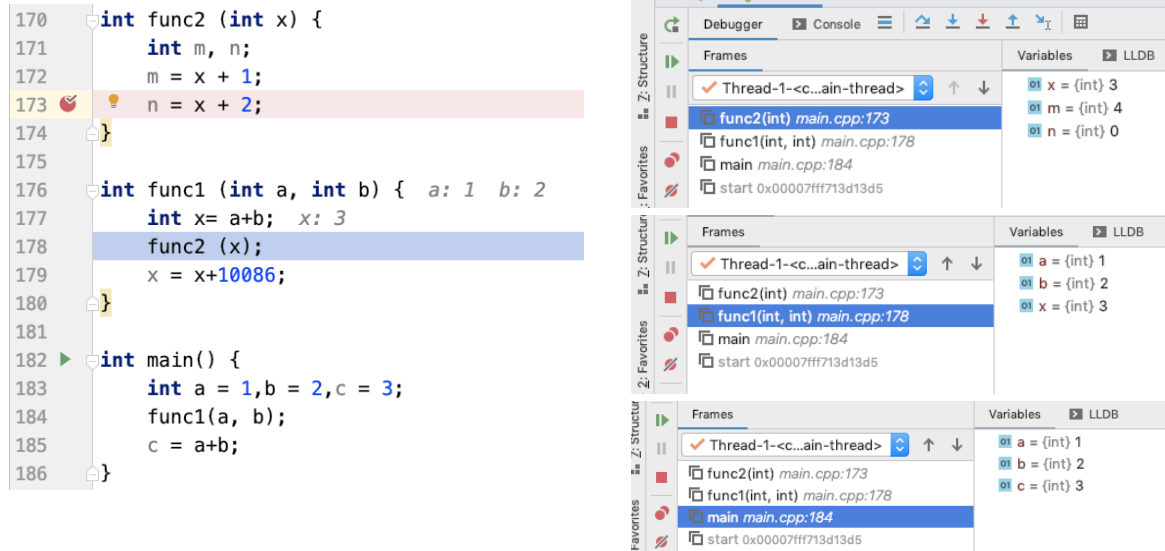
.....

函数调用栈

王道考研/CSKAOYAN.COM

2

函数调用背后的过程



王道考研/CSKAOYAN.COM

3

栈在递归中的应用

适合用“递归”算法解决：可以把原始问题转换为属性相同，但规模较小的问题

Eg 1: 计算正整数的阶乘 $n!$

$$\text{factorial}(n) = \begin{cases} n * \text{factorial}(n-1), & n > 1 \\ 1, & n = 1 \\ 1, & n = 0 \end{cases}$$

递归表达式 (递归体)

边界条件 (递归出口)

Eg 2: 求斐波那契数列

$$\text{Fib}(n) = \begin{cases} \text{Fib}(n-1) + \text{Fib}(n-2), & n > 1 \\ 1, & n = 1 \\ 0, & n = 0 \end{cases}$$

王道考研/CSKAOYAN.COM

4

栈在递归中的应用

Eg 1: 递归算法求阶乘

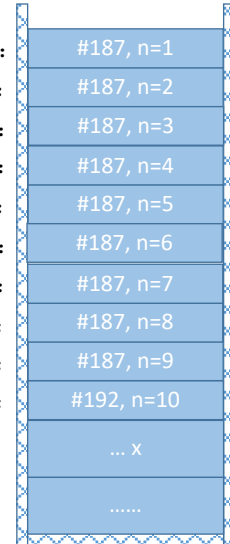
```

182 //计算正整数 n!
183 int factorial (int n){
184     if (n==0 || n==1)
185         return 1;
186     else
187         return n*factorial(n-1);
188 }
189
190 int main() {
191     //... 其他代码
192     int x=factorial(10);
193     printf("奥利给! ");
194 }

```

再次思考：递归算法的空间复杂度

(第10层): #187, n=1
 (第9层): #187, n=2
 (第8层): #187, n=3
 (第7层): #187, n=4
 (第6层): #187, n=5
 (第5层): #187, n=6
 (第4层): #187, n=7
 (第3层): #187, n=8
 (第2层): #187, n=9
 (第1层): #192, n=10
 main: ... x



函数调用栈

递归调用时，函数调用栈可称为“递归工作栈”
每进入一层递归，就将递归调用所需信息压入栈顶
每退出一层递归，就从栈顶弹出相应信息

缺点：太多层递归可能会导致栈溢出

王道考研/CSKAOYAN.COM

5

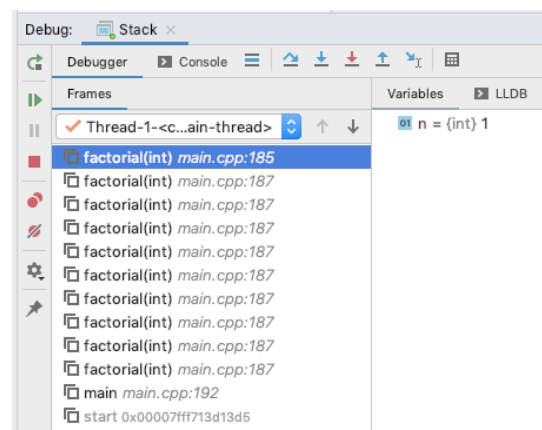
栈在递归中的应用

Eg 1: 递归算法求阶乘

```

182 //计算正整数 n!
183 int factorial (int n){
184     if (n==0 || n==1)
185         return 1;
186     else
187         return n*factorial(n-1);
188 }
189
190 int main() {
191     //... 其他代码
192     int x=factorial(10);
193     printf("奥利给! ");
194 }

```



可以自定义栈将递归算法改造成非递归算法

王道考研/CSKAOYAN.COM

6

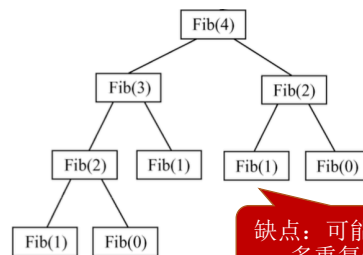
栈在递归中的应用

Eg 2: 递归算法求斐波那契数列

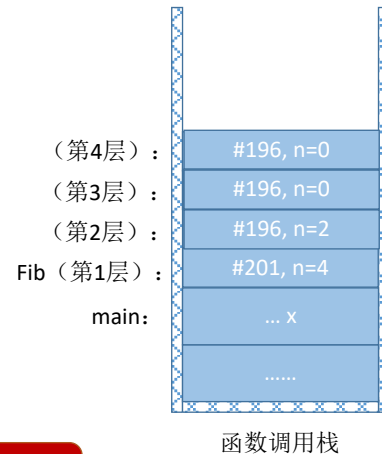
```

190 int Fib(int n){
191     if(n==0)
192         return 0;
193     else if(n==1)
194         return 1;
195     else
196         return Fib(n-1)+Fib(n-2);
197 }
198
199 int main() {
200     //... 其他代码
201     int x=Fib(4);
202     printf("奥利给! ");
203 }

```



缺点：可能包含很多重复计算



王道考研/CSKAOYAN.COM

7

知识回顾与重要考点

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用“函数调用栈”存储：

- ① 调用返回地址
- ② 实参
- ③ 局部变量

递归调用时，函数调用栈可称为“递归工作栈”
每进入一层递归，就将递归调用所需信息压入栈顶
每退出一层递归，就从栈顶弹出相应信息

缺点：效率低，太多层递归可能会导致栈溢出；可能包含很多重复计算

可以自定义栈将递归算法改造成非递归算法

王道考研/CSKAOYAN.COM

8

本节内容

队列
的应用

王道考研/CSKAOYAN.COM

1

队列应用——树的层次遍历

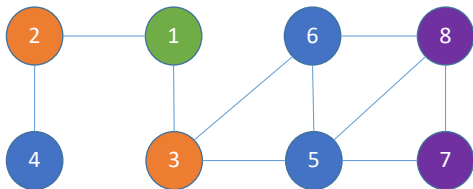
注：在“树”章节中会详细学习

```
graph TD; 1((1)) --- 2((2)); 1 --- 3((3)); 2 --- 4((4)); 2 --- 5((5)); 3 --- 6((6)); 3 --- 7((7)); 5 --- 8((8)); 5 --- 9((9)); 7 --- 10((10)); 7 --- 11((11));
```

王道考研/CSKAOYAN.COM

2

队列应用——图的广度优先遍历



注：在“图”章节中会详细学习

3

队列在操作系统中的应用

多个进程争抢着使用有限的系统资源时，FCFS（First Come First Service，先来先服务）是一种常用策略。

可用队列实现

Eg: CPU资源的分配



就绪进程队列



4

队列在操作系统中的应用

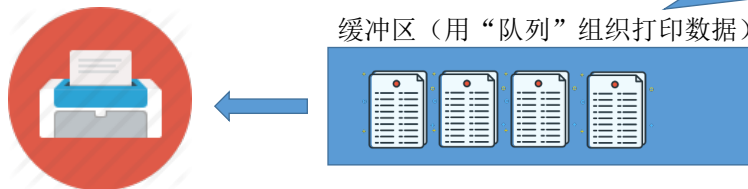
多个进程争抢着使用有限的系统资源时，**FCFS**（First Come First Service，**先来先服务**）是一种常用策略。

可用队列实现

Eg: 打印数据缓冲区

场景：去学校打印店打印论文，多个同学用同一台打印机打印，打印的先后顺序如何？

可缓解主机与打印机
速度不匹配的问题



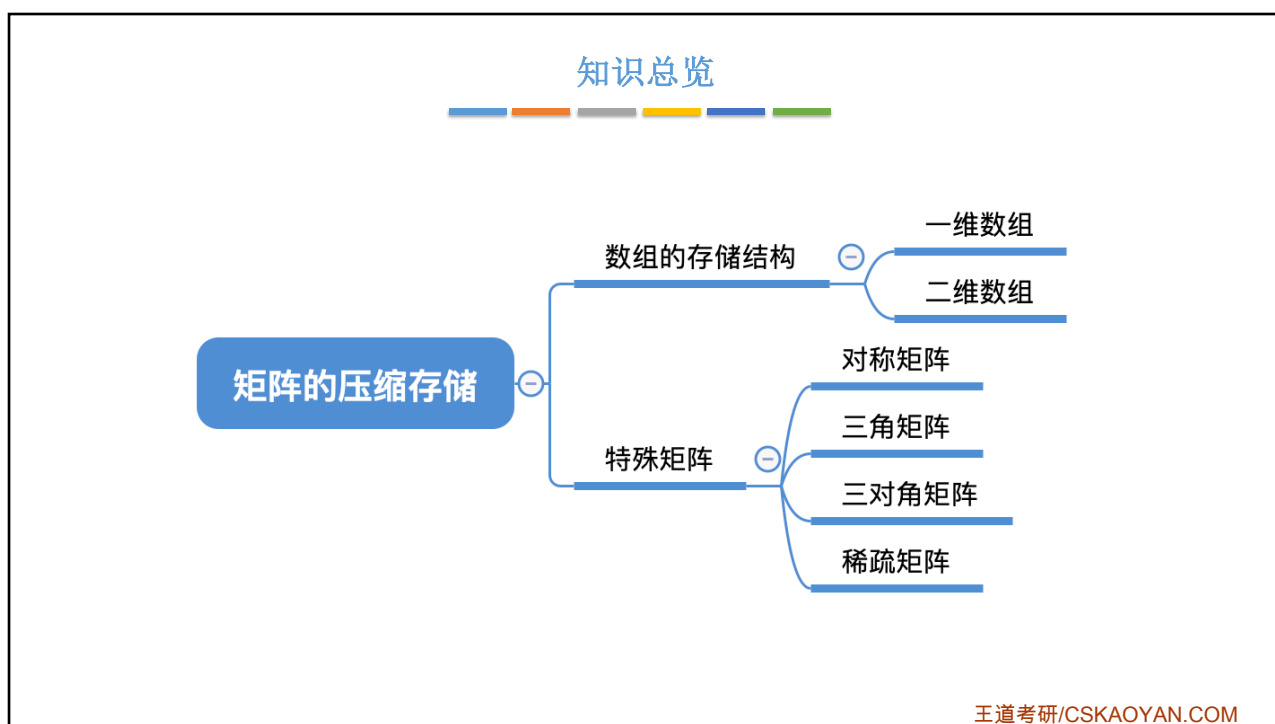
王道考研/CSKAOYAN.COM

本节内容

特殊矩阵
压缩存储

王道考研/CSKAOYAN.COM

1

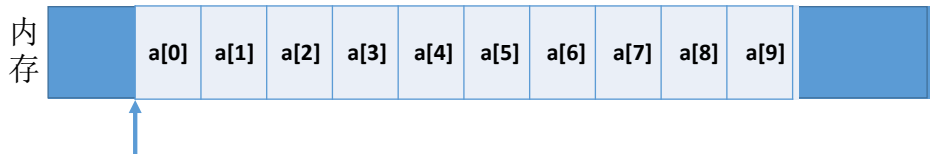


2

一维数组的存储结构

`ElemType a[10];` //ElemType型一维数组

C语言定义
一维数组



起始地址: LOC

各数组元素大小相同, 且物理上连续存放。

数组元素 $a[i]$ 的存放地址 = $LOC + i * \text{sizeof}(\text{ElemType})$ ($0 \leq i < 10$)

注: 除非题目特别说明, 否则数组下标默认从0开始

注意审题!
易错!

王道考研/CSKAOYAN.COM

3

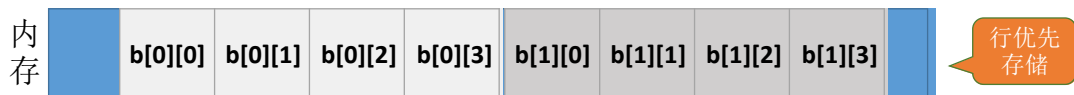
二维数组的存储结构

`ElemType b[2][4];` //2行4列的二维数组

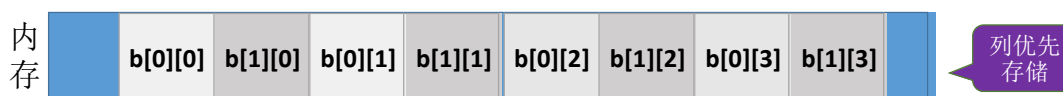
C语言定义
二维数组

$b[0][0]$	$b[0][1]$	$b[0][2]$	$b[0][3]$
$b[1][0]$	$b[1][1]$	$b[1][2]$	$b[1][3]$

逻辑视角



行优先
存储



列优先
存储

王道考研/CSKAOYAN.COM

4

二维数组的存储结构

ElemType b[2][4]; //2行4列的二维数组

C语言定义二维数组

b[0][0]	b[0][1]	b[0][2]	b[0][3]
b[1][0]	b[1][1]	b[1][2]	b[1][3]

逻辑视角

内存

起始地址：LOC

b[0][0]	b[0][1]	b[0][2]	b[0][3]	b[1][0]	b[1][1]	b[1][2]	b[1][3]
---------	---------	---------	---------	---------	---------	---------	---------

行优先存储

M行N列的二维数组 b[M][N] 中，若按行优先存储，则
b[i][j] 的存储地址 = LOC + (i*N + j) * sizeof(ElemType)

王道考研/CSKAOYAN.COM

5

二维数组的存储结构

ElemType b[2][4]; //2行4列的二维数组

C语言定义二维数组

b[0][0]	b[0][1]	b[0][2]	b[0][3]
b[1][0]	b[1][1]	b[1][2]	b[1][3]

逻辑视角

内存

起始地址：LOC

b[0][0]	b[1][0]	b[0][1]	b[1][1]	b[0][2]	b[1][2]	b[0][3]	b[1][3]
---------	---------	---------	---------	---------	---------	---------	---------

列优先存储

M行N列的二维数组 b[M][N] 中，若按列优先存储，则
b[i][j] 的存储地址 = LOC + (j*M + i) * sizeof(ElemType)

王道考研/CSKAOYAN.COM

6

王道考研/CSKAOYAN.COM

3

普通矩阵的存储

$$\begin{matrix}
 a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n-1} & a_{1,n} \\
 a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n-1} & a_{2,n} \\
 a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,n-1} & a_{3,n} \\
 \vdots & \vdots & \vdots & & \vdots & \vdots \\
 a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n-1} & a_{m,n}
 \end{matrix}$$

特殊矩阵

对称矩阵

三角矩阵

三对角矩阵

稀疏矩阵

都是方阵

可用二维数组存储

注意: 描述矩阵元素时, 行、列号通常从 1 开始; 而描述数组时通常下标从 0 开始

(具体看题目给的条件, 注意审题!)

某些特殊矩阵可以压缩存储空间

王道考研/CSKAOYAN.COM

7

对称矩阵的压缩存储

$$\begin{matrix}
 a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n-1} & a_{1,n} \\
 a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n-1} & a_{2,n} \\
 a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,n-1} & a_{3,n} \\
 \vdots & \vdots & \vdots & & \vdots & \vdots \\
 a_{n-1,1} & a_{n-1,2} & a_{n-1,3} & \cdots & a_{n-1,n-1} & a_{n-1,n} \\
 a_{n,1} & a_{n,2} & a_{n,3} & \cdots & a_{n,n-1} & a_{n,n}
 \end{matrix}$$

若 n 阶方阵中任意一个元素 a_{ij} 都有 $a_{ij} = a_{ji}$, 则该矩阵为对称矩阵

普通存储: $n \times n$ 二维数组

压缩存储策略: 只存储主对角线+下三角区 (或主对角线+上三角区)

下三角区: $i > j$

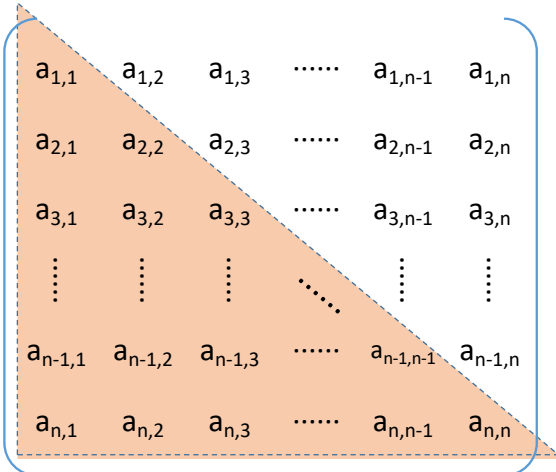
上三角区: $i < j$

主对角线: $i = j$

王道考研/CSKAOYAN.COM

8

对称矩阵的压缩存储



策略: 只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

B[0] B[1] B[2] B[3] B[?]

a _{1,1}	a _{2,1}	a _{2,2}	a _{3,1}		a _{n,n-1}	a _{n,n}
------------------	------------------	------------------	------------------	-------	--------------------	------------------



思考:

- ① 数组大小应为多少?
- ② 站在程序员的角度, 对称矩阵压缩存储后怎样才能方便使用?

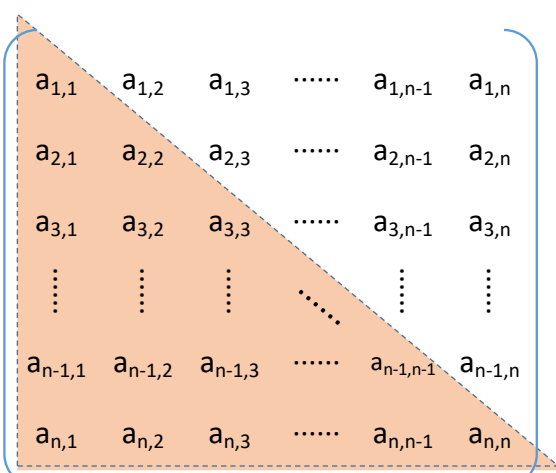
① $(1+n)*n/2$

② 可以实现一个“映射”函数
矩阵下标 → 一维数组下标

王道考研/CSKAOYAN.COM

9

对称矩阵的压缩存储



策略: 只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

B[0] B[1] B[2] B[3] B[$\frac{n(n+1)}{2}-1$]

a _{1,1}	a _{2,1}	a _{2,2}	a _{3,1}		a _{n,n-1}	a _{n,n}
------------------	------------------	------------------	------------------	-------	--------------------	------------------



矩阵下标 → 一维数组下标

$a_{i,j} (i \geq j) \rightarrow B[k]$

key: 按行优先的原则, $a_{i,j}$ 是第几个元素?

$[1+2+\dots+(i-1)] + j \rightarrow$ 第 $\frac{i(i-1)}{2} + j$ 个元素

$\rightarrow k = \frac{i(i-1)}{2} + j - 1$

王道考研/CSKAOYAN.COM

10

王道考研, cskaoyan.com

5

对称矩阵的压缩存储

策略: 只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

$B[0] \quad B[1] \quad B[2] \quad B[3] \quad \dots \quad B[\frac{n(n+1)}{2}-1]$

$a_{1,1}$	$a_{2,1}$	$a_{2,2}$	$a_{3,1}$		$a_{n,1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-------	-----------	-----------

矩阵下标 $\rightarrow$ 一维数组下标

$a_{i,j} \quad (i < j) \rightarrow B[k]$

$a_{i,j} = a_{j,i}$ (对称矩阵性质)

$\rightarrow k = \frac{j(j-1)}{2} + i - 1$

王道考研/CSKAOYAN.COM

11

对称矩阵的压缩存储

策略: 只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

$B[0] \quad B[1] \quad B[2] \quad B[3] \quad \dots \quad B[\frac{n(n+1)}{2}-1]$

$a_{1,1}$	$a_{2,1}$	$a_{2,2}$	$a_{3,1}$		$a_{n,1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-------	-----------	-----------

矩阵下标 $\rightarrow$ 一维数组下标

$a_{i,j} \rightarrow B[k]$

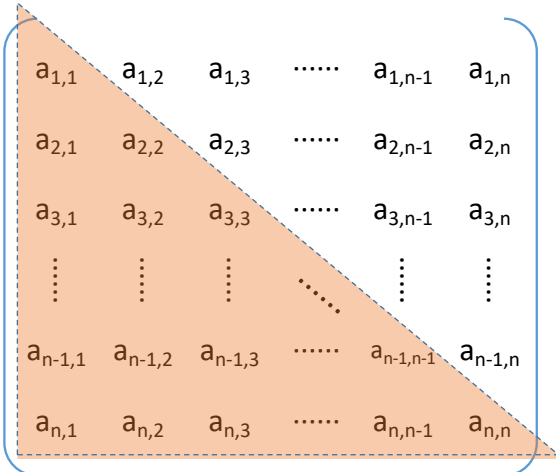
$a_{i,j} = a_{j,i}$ (对称矩阵性质)

$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{j(j-1)}{2} + i - 1, & i < j \text{ (上三角区元素 } a_{i,j} = a_{j,i} \text{)} \end{cases}$

王道考研/CSKAOYAN.COM

12

对称矩阵的压缩存储



策略: 只存储主对角线+下三角区

按**列优先**原则将各元素存入一维数组中。

$B[0] \quad B[1] \quad B[2] \quad B[3] \quad \dots \quad B[\frac{n(n+1)}{2}-1]$

$a_{1,1}$	$a_{2,1}$	$a_{3,1}$	$a_{4,1}$		$a_{n,n-1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-------	-------------	-----------

矩阵下标 $\rightarrow$ 一维数组下标
 $a_{i,j} \rightarrow B[k]$

$a_{i,j} = a_{j,i}$ (对称矩阵性质)

存储上三角? 下三角?

行优先? 列优先?

矩阵元素的下标从0? 1? 开始

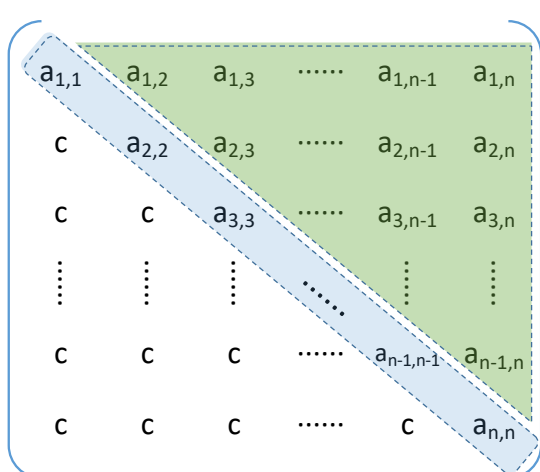
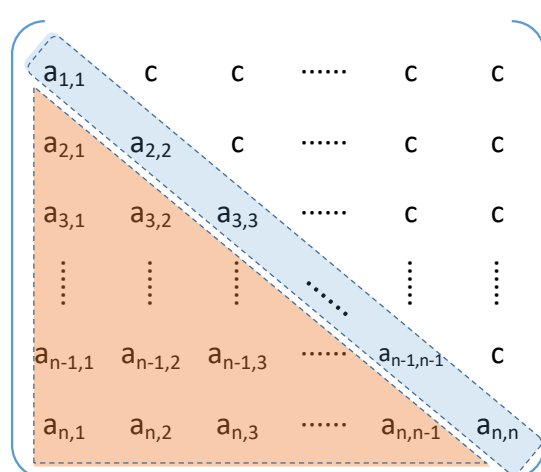
数组下标从0? 1? 开始

出题方法

王道考研/CSKAOYAN.COM

13

三角矩阵的压缩存储



下三角矩阵: 除了主对角线和下三角区, 其余的元素都相同

上三角矩阵: 除了主对角线和上三角区, 其余的元素都相同

王道考研/CSKAOYAN.COM

14

三角矩阵的压缩存储

下三角矩阵: 除了主对角线和下三角区, 其余的元素都相同

压缩存储策略: 按行优先原则将橙色区元素存入一维数组中。并在最后一个位置存储常量c

B[0] B[1] B[2] B[3] ... B[$\frac{n(n+1)}{2}-1$] B[$\frac{n(n+1)}{2}$]

a _{1,1}	a _{2,1}	a _{2,2}	a _{3,1}		a _{n,n-1}	a _{n,n}	c
------------------	------------------	------------------	------------------	-------	--------------------	------------------	---

矩阵下标 → 一维数组下标
a_{i,j} (i ≥ j) → B[k]

Key: 按行优先的原则, a_{i,j} 是第几个元素?

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i < j \text{ (上三角区元素)} \end{cases}$$

王道考研/CSKAOYAN.COM

15

三角矩阵的压缩存储

上三角矩阵: 除了主对角线和上三角区, 其余的元素都相同

压缩存储策略: 按行优先原则将绿色区元素存入一维数组中。并在最后一个位置存储常量c

B[0] B[1] B[2] B[3] ... B[$\frac{n(n+1)}{2}-1$] B[$\frac{n(n+1)}{2}$]

a _{1,1}	a _{1,2}	a _{1,3}	a _{1,4}		a _{1,n}	c
------------------	------------------	------------------	------------------	-------	------------------	---

矩阵下标 → 一维数组下标
a_{i,j} (i ≤ j) → B[k]

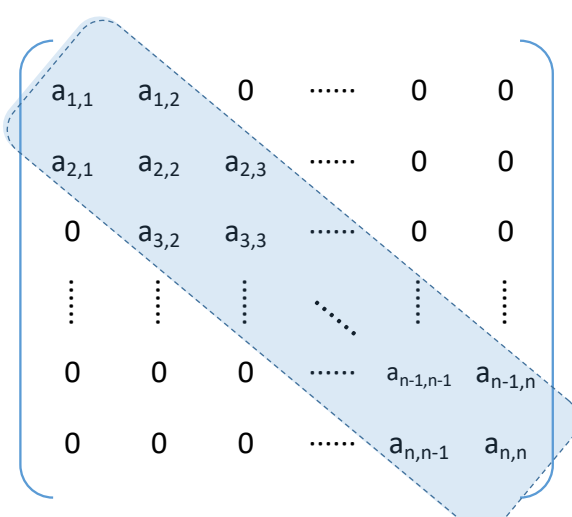
Key: 按行优先的原则, a_{i,j} 是第几个元素?

$$k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i), & i \leq j \text{ (上三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i > j \text{ (下三角区元素)} \end{cases}$$

王道考研/CSKAOYAN.COM

16

三对角矩阵的压缩存储



三对角矩阵, 又称**带状矩阵**:
当 $|i-j|>1$ 时, 有 $a_{ij}=0$ ($1\leq i,j\leq n$)

压缩存储策略:
按**行优先**(或列优先)原则, 只存储带状部分

B[0]	B[1]	B[2]	B[3]	...	B[3n-3]	
$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	...	$a_{n,n-1}$	$a_{n,n}$

矩阵下标 $\rightarrow$ 一维数组下标
 a_{ij} ($|i-j|\leq 1$) $\rightarrow$ B[k]

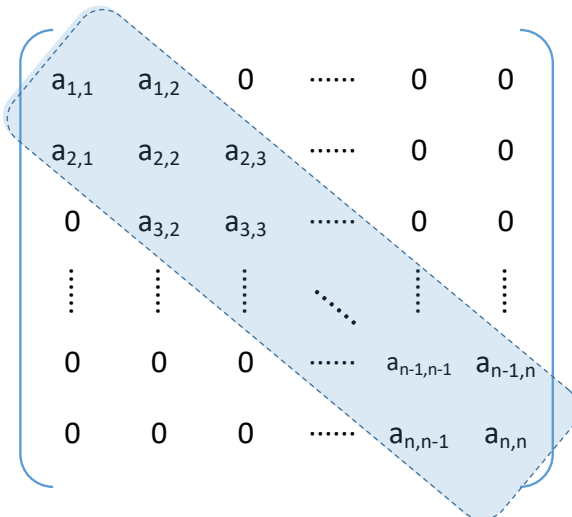
Key: 按**行优先**的原则, a_{ij} 是第几个元素?
前*i*-1行共 $3(i-1)-1$ 个元素
 a_{ij} 是 *i* 行第 $j-i+2$ 个元素
 a_{ij} 是第 $2i+j-2$ 个元素 $\rightarrow$ **$k = 2i+j-3$**

数组下标从0开始

王道考研/CSKAOYAN.COM

17

三对角矩阵的压缩存储



若已知数组下标k, 如何得到 i, j ?
B[k] $\rightarrow$ a_{ij}

第 k+1 个元素, 在第几行? 第几列?
前*i*-1行共 $3(i-1)-1$ 个元素
前*i*行共 $3i-1$ 个元素
显然, $3(i-1)-1 < k+1 \leq 3i-1$

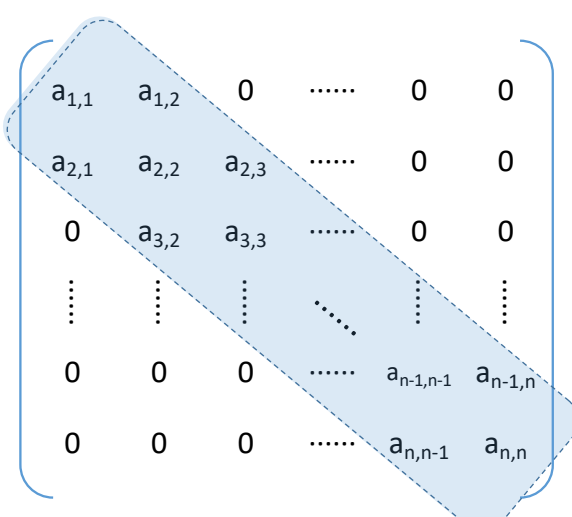
$i \geq (k+2)/3$ 可以理解为“刚好”大于等于
 $i = \lceil (k+2)/3 \rceil$ 向上取整即可满足“刚好”大于等于

王道书的计算逻辑: $3(i-1)-1 \leq k < 3i-1$
 $i \leq (k+1)/3+1$ 可以理解为“刚好”小于等于
 $i = \lfloor (k+1)/3+1 \rfloor$ 向下取整即可满足“刚好”小于等于

王道考研/CSKAOYAN.COM

18

三对角矩阵的压缩存储





若已知数组下标k, 如何得到 i, j ?
 $B[k] \rightarrow a_{i,j}$

第 k+1 个元素, 在第几行? 第几列?

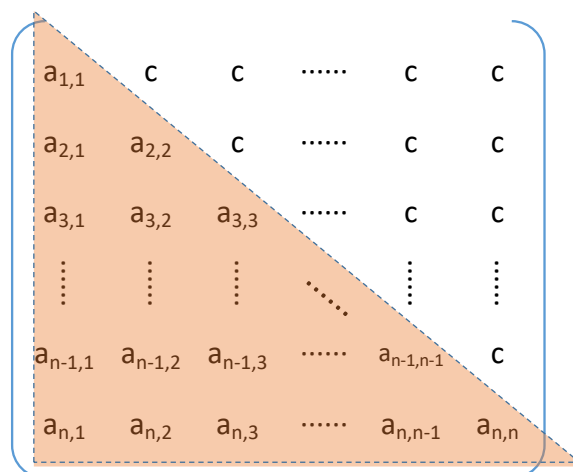
$i = \lfloor (k+2)/3 \rfloor$ 或 $i = \lfloor (k+1)/3 + 1 \rfloor$

由 $k = 2i + j - 3$, 得
 $j = k - 2i + 3$

王道考研/CSKAOYAN.COM

19

三角矩阵的压缩存储



压缩存储策略: 按行优先原则将绿色区元素存入一维数组中。并在最后一个位置存储常量c

$B[0] \ B[1] \ B[2] \ B[3] \ \dots \ B[\frac{n(n+1)}{2}-1] \ B[\frac{n(n+1)}{2}]$

$a_{1,1}$	$a_{2,1}$	$a_{2,2}$	$a_{3,1}$		$a_{n,n}$	c
-----------	-----------	-----------	-----------	-------	-----------	---



矩阵下标 $\rightarrow$ 一维数组下标
 $a_{i,j} \ (i \geq j) \rightarrow B[k]$

Key: 按行优先的原则, $a_{i,j}$ 是第几个元素?

思考: 如何用k推出 i, j?

$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i < j \text{ (上三角区元素)} \end{cases}$

王道考研/CSKAOYAN.COM

20

稀疏矩阵的压缩存储

0

0

4

0

0

5

0

3

0

9

0

0

0

0

0

0

7

0

0

2

0

0

0

0

0

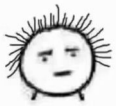
0

0

0

0

0



青少年

稀疏矩阵: 非零元素远远少于矩阵元素的个数

压缩存储策略:
顺序存储——三元组 <行, 列, 值>

i (行)	j (列)	v (值)
1	3	4
1	6	5
2	2	3
2	4	9
3	5	7
4	2	2

(注: 此处行、列标从1开始)

王道考研/CSKAOYAN.COM

21

稀疏矩阵的压缩存储

0

0

4

0

0

5

0

3

0

9

0

0

0

0

0

0

7

0

0

2

0

0

0

0

0

0

0

0

0

0

非零数据

行	列	值
1	3	4
1	6	5
2	2	3
2	4	9
3	5	7
4	2	2

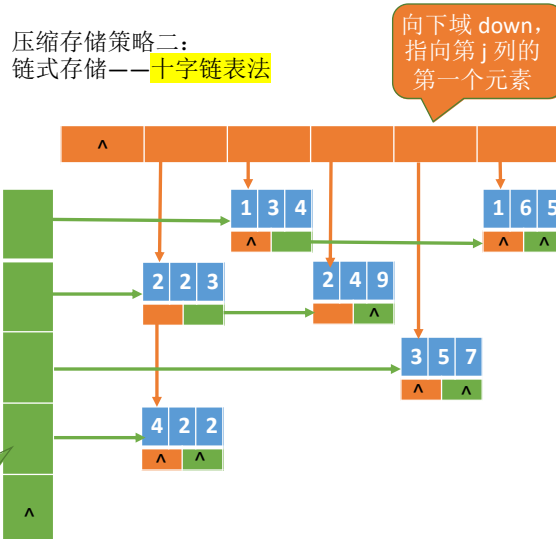
结点说明:

指向同列的下一个元素	指向同行的下一个元素
1	1
1	1
1	1
1	1
1	1
1	1

压缩存储策略二:
链式存储——十字链表法

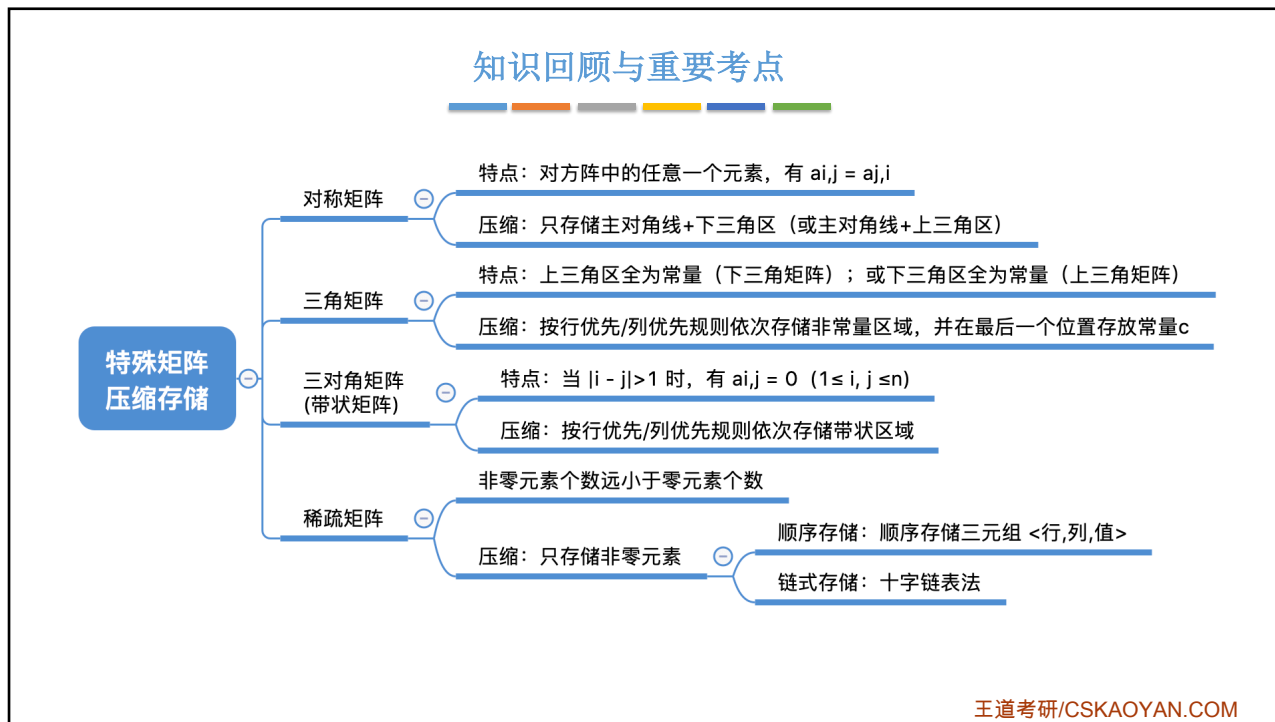
向下域 down, 指向第 j 列的第一个元素

向右域 right, 指向第 i 行的第一个元素

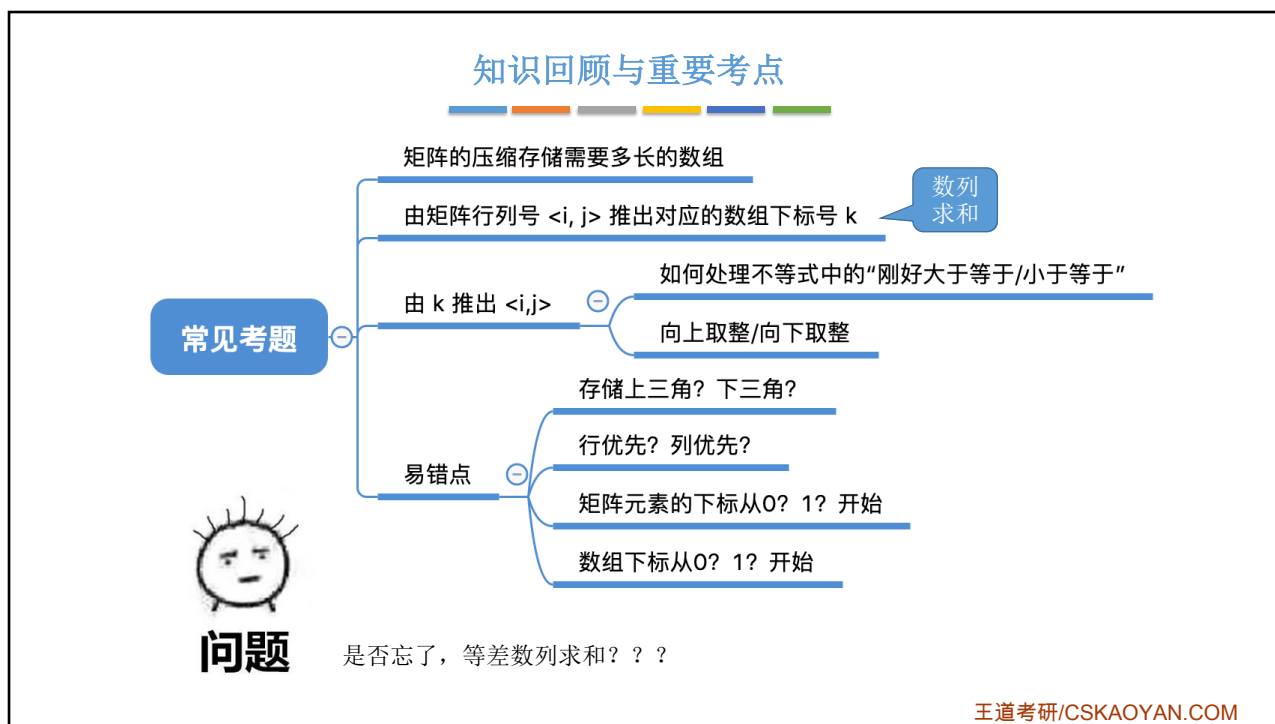


王道考研/CSKAOYAN.COM

22



23



24